



UNIVERSITÉ SULTAN MOULAY SLIMANE

Faculté des Sciences et Techniques

Béni-Mellal

Master : Intelligence Artificielle et Digital Computing

Projet En IA

Handwritten Digit Recognition

using Machine and Deep Learning Algorithms

Réalisé par :
ENNACIRI Rabab

Encadré par :
Pr. Mourad Nachaoui

Année universitaire 2025-2026

Résumé

La reconnaissance automatique des chiffres manuscrits constitue l'un des problèmes les plus étudiés dans le domaine de l'apprentissage automatique et de la vision par ordinateur. L'objectif de ce travail est de comparer les performances de trois approches de classification largement utilisées, à savoir le Support Vector Machine (SVM), le Multilayer Perceptron (MLP) et le Convolutional Neural Network (CNN), appliquées à la base de données MNIST.

Dans un premier temps, les données ont subi plusieurs étapes de prétraitement, notamment la normalisation des pixels, le redimensionnement des images et l'encodage One-Hot des classes. Des techniques d'augmentation de données ont également été utilisées afin d'améliorer la capacité de généralisation des modèles de réseaux de neurones.

Par la suite, les trois modèles ont été implémentés et optimisés à l'aide de différentes stratégies telles que la réduction de dimension par ACP (PCA) pour le SVM, la régularisation L2, le Batch Normalization, le Dropout, le Label Smoothing ainsi que les mécanismes Early Stopping et ReduceLROnPlateau pour les architectures MLP et CNN. Ces améliorations ont permis d'augmenter les performances tout en conservant les principes fondamentaux de chaque modèle.

Les résultats expérimentaux montrent que le modèle CNN offre les meilleures performances avec une précision de test supérieure à 99 %, confirmant sa capacité à extraire automatiquement les caractéristiques spatiales des images. Le MLP obtient également des résultats satisfaisants grâce aux différentes techniques de régularisation utilisées, tandis que le SVM conserve de bonnes performances malgré une capacité de représentation plus limitée pour les données visuelles complexes.

Cette étude met en évidence l'efficacité des méthodes de Deep Learning pour la reconnaissance des chiffres manuscrits et démontre l'impact positif des techniques d'optimisation et de régularisation sur les performances des modèles de classification.

Sommaire

Liste des figures **6**

1 Cadre Général du Projet **9**

Cadre Général du Projet **9**

1.1 Introduction 9
1.2 Contexte de l'étude 9
1.3 Problématique 9
1.4 Objectifs 10
1.5 Conclusion 11

2 Méthodologie de résolution **12**

Cadre Général du Projet **12**

2.1 Introduction 12
2.2 Base de données MNIST 12
2.3 Prétraitement des données 13
2.3.1 Redimensionnement des images 13
2.3.2 Conversion en format `float32` et normalisation 14
2.3.3 Encodage One-Hot des classes 15
2.4 Algorithmes étudiés 15
2.4.1 Support Vector Machine (SVM) 15
2.4.1.1 Principe de fonctionnement 16
2.4.1.2 Vecteurs de support 16
2.4.1.3 Maximisation de la marge 17
2.4.1.4 SVM à marge souple 17
2.4.1.5 SVM linéaire et SVM non linéaire 18
2.4.1.6 Classification multi-classes : stratégie One-vs-Rest 19
2.4.2 Implémentation du SVM dans l'article 19
2.4.3 Multilayer Perceptron (MLP) 22
2.4.3.1 Architecture générale du MLP 22
2.4.3.2 Modèle mathématique d'un neurone 23
2.4.3.3 Propagation avant (Forward Propagation) 23
2.4.3.4 Fonctions d'activation 24
2.4.3.5 Fonction de coût 25
2.4.3.6 Rétropropagation du gradient (Backpropagation) 25
2.4.4 Implémentation du MLP dans l'article 26

2.4.4.1 Propagation dans les couches cachées . . . 27
2.4.4.2 Fonction d'activation ReLU . . . 27
2.4.4.3 Régularisation par Dropout . . . 27
2.4.4.4 Couche de sortie Softmax . . . 28
2.4.4.5 Fonction de perte . . . 29
2.4.4.6 Apprentissage par rétropropagation . . . 29
2.4.5 Convolutional Neural Network (CNN) . . . 29
2.4.5.1 Principe général d'un CNN . . . 30
2.4.5.2 Représentation mathématique des images . . . 30
2.4.5.3 Couche de convolution . . . 31
2.4.5.4 Paramètres de convolution . . . 31
2.4.5.5 Fonction d'activation ReLU . . . 32
2.4.5.6 Couche de Pooling . . . 32
2.4.5.7 Extraction hiérarchique des caractéristiques . . . 33
2.4.5.8 Transformation Flatten . . . 33
2.4.5.9 Couches Fully Connected . . . 34
2.4.5.10 Fonction Softmax . . . 34
2.4.5.11 Fonction de perte . . . 34
2.4.5.12 Rétropropagation des erreurs . . . 35
2.4.5.13 Avantages des CNN . . . 35
2.4.6 Implémentation du CNN dans l'article . . . 36
2.4.6.1 Représentation des images d'entrée . . . 36
2.4.6.2 Première couche de convolution . . . 37
2.4.6.3 Dimension de sortie de la première convolution . . . 38
2.4.6.4 Fonction d'activation ReLU . . . 38
2.4.6.5 Deuxième couche de convolution . . . 38
2.4.6.6 Couche de Max Pooling . . . 39
2.4.6.7 Régularisation par Dropout . . . 39
2.4.6.8 Couche Flatten . . . 40
2.4.6.9 Couche entièrement connectée . . . 40
2.4.6.10 Deuxième couche Dropout . . . 41
2.4.6.11 Couche de sortie Softmax . . . 41
2.5 Environnement expérimental . . . 42
2.5.1 Python . . . 42
2.5.2 Google Colab . . . 42
2.5.3 NumPy . . . 43
2.5.4 Scikit-Learn . . . 43
2.5.5 TensorFlow . . . 43
2.5.6 Keras . . . 44

2.6 Améliorations proposées 44
2.6.1 Amélioration du SVM 44
2.6.1.1 Principe mathématique du SVM 46
2.6.1.2 Utilisation du noyau RBF 47
2.6.1.3 Paramètre C 47
2.6.1.4 Paramètre γ 48
2.6.2 Amélioration du MLP 48
2.6.2.1 Architecture du MLP 49
2.6.2.2 Couche Flatten 50
2.6.2.3 Fonction d'activation ReLU 51
2.6.2.4 Batch Normalization 51
2.6.2.5 Régularisation par Dropout 52
2.6.2.6 Régularisation L2 53
2.6.2.7 Couche de sortie Softmax 54
2.6.2.8 Label Smoothing 55
2.6.2.9 Fonction de perte : Categorical Cross Entropy 56
2.6.2.10 Optimiseur Adam 57
2.6.2.11 Early Stopping et ReduceLROnPlateau 58
2.6.3 Amélioration du CNN 59
2.6.3.1 Augmentation artificielle des données 59
2.6.3.2 Architecture du réseau CNN proposé 60
2.6.3.3 Premier bloc convolutionnel 60
2.6.3.4 Première couche de Max Pooling 60
2.6.3.5 Première couche Dropout 61
2.6.3.6 Deuxième bloc convolutionnel 61
2.6.3.7 Deuxième couche de Max Pooling 62
2.6.3.8 Deuxième couche Dropout 62
2.6.3.9 Couche Flatten 62
2.6.3.10 Couche Dense cachée 62
2.6.3.11 Couche de sortie 63
2.6.3.12 Fonction de perte 63
2.6.3.13 Optimiseur Adam 64
2.6.3.14 Early Stopping 64
2.6.3.15 Réduction adaptative du taux d'apprentissage 64
2.7 Conclusion 65

3 Méthodologie d'évaluation **66**

Méthodologie d'évaluation **66**
3.1 Introduction 66
3.2 Métriques d'évaluation 66
3.2.1 Accuracy 66
3.2.2 Temps d'exécution 67
3.3 Résultats des modèles implémentés selon l'article 67
3.3.1 Évolution de l'accuracy du modèle CNN 67
3.4 Résultats après amélioration des modèles 68
3.5 Conclusion 72

Liste des figures

1 Architecture fonctionnelle du classifieur SVM linéaire appliqué à la base MNIST. 19
2 Architecture du modèle MLP pour la reconnaissance de chiffres manuscrits (MNIST) 26
3 Architecture du modèle CNN pour la reconnaissance de chiffres manuscrits (MNIST) 36
4 Python 42
5 Google Colab 42
6 NumPy 43
7 Scikit-Learn 43
8 TensorFlow 43
9 Keras 44
10 Évolution de l'accuracy d'entraînement et de test du modèle CNN 68
11 Évolution de l'accuracy d'entraînement et de test du modèle CNN 70
12 Évolution de l'accuracy d'entraînement et de test du modèle MLP 71
13 MATRICE DE CONFUSION 72

Introduction Générale

L'intelligence artificielle et l'apprentissage automatique connaissent aujourd'hui un développement considérable dans de nombreux domaines tels que la vision par ordinateur, le traitement du langage naturel, la médecine, la finance et les systèmes intelligents. Parmi les applications les plus importantes de ces technologies figure la reconnaissance automatique des caractères manuscrits, qui représente un problème fondamental de classification d'images.

La reconnaissance des chiffres manuscrits consiste à identifier automatiquement le chiffre représenté dans une image numérique. Cette problématique est largement étudiée dans la littérature scientifique en raison de ses nombreuses applications pratiques, notamment la lecture automatique des chèques bancaires, la numérisation de documents administratifs, la reconnaissance de formulaires et les systèmes de tri postal. Cependant, les variations d'écriture entre individus rendent cette tâche complexe et nécessitent l'utilisation de modèles d'apprentissage performants capables de généraliser correctement sur de nouvelles données.

Afin d'évaluer les performances des différentes approches de classification, la base de données MNIST est devenue une référence incontournable dans le domaine de l'apprentissage automatique. Cette base contient 70 000 images de chiffres manuscrits en niveaux de gris, réparties en 60 000 images d'entraînement et 10 000 images de test. Grâce à sa popularité, elle permet de comparer efficacement différents algorithmes de classification dans des conditions identiques.

Dans ce contexte, plusieurs techniques d'apprentissage automatique et d'apprentissage profond ont été développées pour résoudre ce problème. Parmi les approches les plus utilisées figurent le Support Vector Machine (SVM), le Multilayer Perceptron (MLP) et le Convolutional Neural Network (CNN). Le SVM constitue une méthode robuste de classification supervisée basée sur la recherche d'un hyperplan optimal séparant les différentes classes. Le MLP représente une architecture de réseau de neurones entièrement connectée capable d'apprendre des relations non linéaires complexes entre les données. Quant au CNN, il est spécialement conçu pour le traitement des images grâce à l'utilisation d'opérations de convolution permettant l'extraction automatique de caractéristiques pertinentes.

L'objectif principal de ce projet est d'étudier, d'implémenter et de comparer ces trois approches sur la base de données MNIST. Dans un premier temps, les modèles décrits dans la littérature ont été analysés afin de comprendre leur fonctionnement théorique et leurs performances. Dans un second temps, plusieurs techniques d'amélioration ont été introduites, notamment la normalisation des données, l'augmentation artificielle du jeu de données (Data Augmentation), la Batch Normalization, le Dropout, la régularisation ainsi que différentes stratégies d'optimisation. Ces améliorations ont permis d'augmenter les performances des modèles tout en conservant leurs principes architecturaux fondamentaux.

Enfin, les performances obtenues sont évaluées et comparées à l'aide de différentes métriques telles que la précision de classification (Accuracy), la fonction de coût (Loss), le temps

d'exécution ainsi que l'analyse des matrices de confusion. Cette étude permet ainsi de mettre en évidence les avantages et les limites de chaque approche pour la reconnaissance des chiffres manuscrits et d'identifier le modèle le plus performant dans le cadre du problème étudié [1].

Chapitre 1 : Cadre Général du Projet

1.1 Introduction

La reconnaissance automatique des chiffres manuscrits constitue l'une des applications les plus importantes de l'apprentissage automatique et de la vision par ordinateur. Grâce aux progrès récents des techniques de Machine Learning et de Deep Learning, plusieurs modèles performants ont été développés pour résoudre ce problème avec une grande précision.

Ce chapitre présente le cadre général de cette étude. Il expose tout d'abord le contexte dans lequel s'inscrit le projet, puis définit la problématique abordée. Enfin, les objectifs poursuivis sont présentés afin de préciser les contributions attendues de ce travail portant sur l'implémentation, l'évaluation et l'amélioration des modèles SVM, MLP et CNN appliqués à la base de données MNIST.

1.2 Contexte de l'étude

La classification d'images représente aujourd'hui l'un des domaines les plus actifs de l'intelligence artificielle. Parmi les problèmes les plus étudiés figure la reconnaissance automatique des chiffres manuscrits, qui constitue une tâche de référence pour l'évaluation des algorithmes d'apprentissage automatique et d'apprentissage profond.

La base de données MNIST est largement utilisée dans ce contexte en raison de sa simplicité et de sa richesse. Elle contient 70 000 images de chiffres manuscrits réparties en dix classes correspondant aux chiffres de 0 à 9. Cette base est devenue un benchmark standard permettant de comparer objectivement les performances de différents modèles de classification.

De nombreux travaux de recherche ont exploité cette base afin d'évaluer l'efficacité de diverses approches telles que les Machines à Vecteurs de Support (SVM), les Perceptrons Multicouches (MLP) et les Réseaux de Neurones Convolutifs (CNN). Les résultats obtenus montrent que les performances dépendent fortement de l'architecture utilisée, des techniques de prétraitement appliquées ainsi que du choix des hyperparamètres.

Dans le cadre de ce projet, une étude comparative de ces trois approches est réalisée sur la base MNIST. En s'inspirant d'un article scientifique de référence, plusieurs améliorations sont apportées aux modèles implémentés afin d'analyser leur impact sur les performances de classification et d'identifier la solution la plus efficace.

1.3 Problématique

Malgré les progrès réalisés dans le domaine de la reconnaissance automatique des chiffres manuscrits, plusieurs défis demeurent. Les variations d'écriture, les différences de taille, les rotations ou encore la qualité des images peuvent affecter les performances des modèles de classification.

Les approches classiques comme le SVM offrent généralement de bonnes performances, mais peuvent être limitées lorsqu'il s'agit de traiter des données complexes ou de grande dimension. Les réseaux de neurones, quant à eux, nécessitent une architecture adaptée ainsi qu'un réglage précis des hyperparamètres afin d'obtenir des résultats optimaux.

La problématique principale de ce projet peut donc être formulée comme suit :

> *« Comment améliorer la reconnaissance automatique des chiffres manuscrits de la base MNIST en utilisant des techniques d'apprentissage automatique et d'apprentissage profond tout en obtenant une précision élevée et une bonne capacité de généralisation ? »*

Pour répondre à cette problématique, plusieurs modèles de classification sont étudiés, implémentés et optimisés afin d'identifier l'approche la plus performante.

1.4 Objectifs

L'objectif principal de ce projet est de développer et d'évaluer différents modèles de classification capables de reconnaître automatiquement les chiffres manuscrits de la base de données MNIST.

Les objectifs spécifiques sont les suivants :

- Charger et préparer la base de données MNIST.
- Réaliser les opérations de prétraitement des données (normalisation, restructuration et encodage).
- Implémenter un classificateur basé sur les Machines à Vecteurs de Support (SVM).
- Concevoir un modèle de Perceptron Multicouche (MLP) amélioré.
- Développer un Réseau de Neurones Convolutif (CNN) optimisé pour la classification des chiffres manuscrits.
- Appliquer plusieurs techniques d'amélioration afin d'augmenter les performances des modèles, notamment :
  - la normalisation des données ;
  - le Data Augmentation ;
  - la Batch Normalization ;
  - le Dropout ;
  - la réduction dynamique du taux d'apprentissage ;
  - l'arrêt anticipé (Early Stopping).
- Comparer les performances des différents modèles à l'aide de plusieurs métriques d'évaluation.
- Analyser les courbes d'apprentissage (Accuracy et Loss).

- Évaluer les modèles en termes de précision de classification, de temps d'exécution et de matrice de confusion.
- Identifier l'algorithme offrant les meilleures performances sur la base de données MNIST.

1.5 Conclusion

Ce chapitre a présenté le cadre général du projet, en exposant le contexte, la problématique et les objectifs poursuivis. La reconnaissance automatique des chiffres manuscrits constitue une tâche de référence permettant d'évaluer et de comparer différentes approches d'apprentissage automatique et profond sur la base de données MNIST. Dans ce cadre, trois modèles ont été étudiés et optimisés : le SVM, le MLP et le CNN, en appliquant plusieurs techniques d'amélioration afin d'accroître leurs performances et leur capacité de généralisation.

Chapitre 2 : Méthodologie de résolution

2.1 Introduction

Après avoir présenté le contexte général du projet ainsi que la problématique étudiée, ce chapitre décrit la méthodologie adoptée pour la reconnaissance des chiffres manuscrits à partir de la base de données MNIST.

La démarche suivie s'appuie sur l'étude et la reproduction des approches proposées dans l'article de référence, notamment les algorithmes Support Vector Machine (SVM), Multilayer Perceptron (MLP) et Convolutional Neural Network (CNN). Une première phase de prétraitement des données est réalisée afin de préparer les images pour l'apprentissage automatique et l'apprentissage profond.

Par la suite, les architectures initialement proposées dans l'article sont présentées et analysées. Afin d'améliorer les performances de classification, plusieurs optimisations ont été introduites au niveau des modèles étudiés. Ces améliorations concernent principalement l'enrichissement des architectures, l'ajout de techniques de régularisation, l'augmentation artificielle des données ainsi que l'optimisation des hyperparamètres d'entraînement.

Ce chapitre est organisé en trois parties principales. La première présente les étapes de prétraitement appliquées aux données. La deuxième décrit les méthodes originales utilisées dans l'article de référence. Enfin, la troisième expose les améliorations proposées ainsi que les modifications apportées aux modèles afin d'obtenir de meilleures performances de classification.

2.2 Base de données MNIST

La base de données MNIST (*Modified National Institute of Standards and Technology*) est l'une des bases de données les plus utilisées dans le domaine de la reconnaissance de caractères manuscrits et de l'apprentissage automatique. Elle constitue une référence standard pour l'évaluation et la comparaison des performances des algorithmes de classification d'images.

MNIST est une version améliorée de la base de données NIST et résulte de la combinaison de deux ensembles de données distincts : *Special Database 1* et *Special Database 3*. Ces bases contiennent des chiffres manuscrits produits respectivement par des étudiants et par des employés du Bureau du recensement des États-Unis. Les données ont ensuite été prétraitées afin d'obtenir un ensemble homogène adapté aux tâches de reconnaissance automatique.

La base MNIST contient un total de 70 000 images de chiffres manuscrits réparties en deux sous-ensembles :

— 60 000 images destinées à l'apprentissage (*Training Set*) ;

— 10 000 images destinées au test (*Test Set*).

Chaque image représente un chiffre compris entre 0 et 9 et possède une résolution fixe de :

$$28 \times 28 = 784 \text{ pixels}$$

Les images sont codées en niveaux de gris, où chaque pixel est représenté par une valeur comprise entre 0 et 255. Une valeur proche de 0 correspond à une couleur noire tandis qu'une valeur proche de 255 correspond à une couleur blanche.

Mathématiquement, chaque image peut être représentée par une matrice :

$$X \in \mathbb{R}^{28\times28}$$

où chaque élément x_{ij} correspond à l'intensité lumineuse du pixel situé à la position (i, j).

À chaque image est associée une étiquette y indiquant la classe réelle du chiffre représenté :

$$y \in \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

L'objectif des modèles étudiés dans ce travail consiste à apprendre une fonction de classification capable d'associer correctement une image d'entrée X à son étiquette correspondante y.

Grâce à sa taille importante, sa diversité et sa standardisation, la base MNIST constitue un environnement de référence particulièrement adapté à l'évaluation des performances des algorithmes SVM, MLP et CNN utilisés dans cette étude.

2.3 Prétraitement des données

Le prétraitement constitue une étape fondamentale dans les systèmes d'apprentissage automatique et d'apprentissage profond. Son objectif est de préparer les données brutes avant leur utilisation par les modèles de classification. Cette phase permet de réduire les incohérences, d'améliorer la qualité des données et de faciliter le processus d'apprentissage. Dans cette étude, plusieurs opérations de prétraitement ont été appliquées à la base de données MNIST, notamment le redimensionnement des images, la conversion des données en format numérique adapté, la normalisation des pixels et l'encodage des classes sous forme catégorielle.

2.3.1 Redimensionnement des images

Les images de la base de données MNIST sont initialement représentées sous forme de matrices de dimensions 28×28, où chaque élément correspond à l'intensité lumineuse d'un pixel en niveau de gris.

Afin de rendre les données compatibles avec les modèles de réseaux de neurones utilisés dans cette étude, chaque image est restructurée sous la forme :

$$28 \times 28 \times 1$$

où :

— 28 représente la hauteur de l'image ;

— 28 représente sa largeur ;

— 1 représente le canal de couleur correspondant au niveau de gris.

Cette transformation conserve l'information spatiale de l'image tout en ajoutant explicitement la dimension du canal, ce qui facilite son traitement par les algorithmes d'apprentissage profond.

2.3.2 Conversion en format `float32` et normalisation

Les pixels de la base MNIST sont codés sur 8 bits et prennent des valeurs entières comprises entre :

$$0 \leq x \leq 255$$

où x représente l'intensité d'un pixel.

Avant d'effectuer la normalisation, les données sont converties vers le format numérique `float32`. Cette opération permet de représenter les valeurs sous forme réelle plutôt que sous forme entière et facilite les calculs réalisés lors de l'apprentissage du modèle.

Mathématiquement, cette transformation peut être représentée par :

$$x \leftarrow \texttt{float32}(x)$$

Après cette conversion, une opération de normalisation est appliquée afin de ramener toutes les valeurs des pixels dans l'intervalle $[0, 1]$. Cette transformation est réalisée selon la formule suivante :

$$x_{norm} = \frac{x}{255}$$

où :

— x représente la valeur initiale du pixel ;

— x_{norm} représente la valeur normalisée.

Après normalisation, les valeurs vérifient :

$$0 \leq x_{norm} \leq 1$$

Cette mise à l'échelle réduit les écarts entre les caractéristiques d'entrée et améliore la stabilité numérique du processus d'apprentissage. Elle favorise également une convergence plus rapide des algorithmes d'optimisation tels que la descente de gradient.

2.3.3 Encodage One-Hot des classes

Les étiquettes de la base MNIST sont initialement représentées par des valeurs entières comprises entre 0 et 9. Cependant, dans un problème de classification multiclasses, les réseaux de neurones nécessitent généralement une représentation vectorielle des classes.

Pour cette raison, un encodage One-Hot est appliqué aux étiquettes. Cette méthode consiste à représenter chaque classe par un vecteur binaire dont la taille est égale au nombre total de classes.

Pour un problème comportant K classes, une étiquette y est transformée en un vecteur :

$$\mathbf{y} = [y_1, y_2, \ldots, y_K]$$

tel que :

$$y_i = \begin{cases} 1 & \text{si } i = y \\ 0 & \text{sinon} \end{cases}$$

Dans le cas de la base MNIST :

$$K = 10$$

car le problème consiste à reconnaître les chiffres de 0 à 9.

Par exemple, l'étiquette correspondant au chiffre 4 est transformée en :

$$[0, 0, 0, 0, 1, 0, 0, 0, 0, 0]$$

Cette représentation permet au modèle d'associer chaque neurone de sortie à une classe spécifique et facilite le calcul de la fonction de perte lors de l'apprentissage.

Ainsi, grâce aux opérations de redimensionnement, de conversion en `float32`, de normalisation et d'encodage One-Hot, les données de la base MNIST sont transformées dans un format optimal pour l'entraînement des modèles de classification étudiés dans ce travail.

2.4 Algorithmes étudiés

2.4.1 Support Vector Machine (SVM)

Le Support Vector Machine (SVM) est un algorithme d'apprentissage supervisé largement utilisé pour les problèmes de classification et de régression. Introduit par Vladimir Vapnik, il repose sur le principe de la séparation optimale des données à l'aide d'un hyperplan. L'objectif principal du SVM est de construire une frontière de décision capable de distinguer les différentes classes tout en maximisant la capacité de généralisation du modèle sur des données non observées.

Considérons un ensemble d'apprentissage composé de N échantillons :

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$$

où $x_i \in \mathbb{R}^d$ représente le vecteur de caractéristiques et $y_i \in \{-1, +1\}$ l'étiquette de classe associée.

2.4.1.1 Principe de fonctionnement

L'idée fondamentale du SVM consiste à rechercher un hyperplan séparateur défini par :

$$w^T x + b = 0$$

où :

— w est le vecteur des poids ;

— b est le biais ;

— x représente un vecteur de caractéristiques.

Le vecteur w détermine l'orientation de l'hyperplan dans l'espace des caractéristiques tandis que le biais b permet de définir sa position.

Pour un nouvel échantillon x, le SVM calcule une fonction de décision donnée par :

$$f(x) = w^T x + b$$

Cette fonction permet de déterminer de quel côté de l'hyperplan se situe l'observation. La règle de classification est donnée par :

$$\hat{y} = sign(f(x))$$

Ainsi :

$$f(x) > 0$$

indique que l'échantillon appartient à la classe positive, alors que :

$$f(x) < 0$$

indique qu'il appartient à la classe négative.

2.4.1.2 Vecteurs de support

Une caractéristique essentielle du SVM est l'utilisation des vecteurs de support (*Support Vectors*). Ces observations correspondent aux points situés au plus près de la frontière de décision. Contrairement à plusieurs algorithmes de classification, seuls ces points influencent directement la construction de l'hyperplan optimal.

La distance géométrique entre un point x et l'hyperplan est donnée par :

$$d(x, H) = \frac{|w^T x + b|}{\|w\|}$$

Cette distance mesure l'éloignement de l'observation par rapport à la frontière de décision.

2.4.1.3 Maximisation de la marge

L'objectif du SVM n'est pas uniquement de séparer correctement les données, mais également de maximiser la marge entre les différentes classes. La marge correspond à la distance séparant les deux hyperplans parallèles :

$$w^T x + b = 1$$

et

$$w^T x + b = -1$$

Les observations situées sur ces deux frontières sont précisément les vecteurs de support.
La largeur de la marge est donnée par :

$$Margin = \frac{2}{\|w\|}$$

Ainsi, maximiser la marge revient à minimiser la norme du vecteur w.
Le problème d'optimisation du SVM linéaire s'écrit :

$$\min_{w,b} \frac{1}{2}\|w\|^2$$

sous la contrainte :

$$y_i(w^T x_i + b) \geq 1$$

pour tout échantillon i.

Cette formulation permet d'obtenir une frontière de décision robuste offrant généralement une bonne capacité de généralisation.

2.4.1.4 SVM à marge souple

Dans les applications réelles, les données sont rarement parfaitement séparables. Certaines observations peuvent être bruitées ou mal positionnées. Afin de prendre en compte ces situations, le SVM introduit des variables de relâchement ξ_i permettant d'autoriser certaines erreurs de classification.

La contrainte devient alors :

$$y_i(w^T x_i + b) \geq 1 - \xi_i$$

avec :

$$\xi_i \geq 0$$

La fonction objectif devient :

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C \sum_{i=1}^{N} \xi_i$$

où C est un paramètre de régularisation.

Ce paramètre contrôle le compromis entre la maximisation de la marge et la réduction des erreurs de classification :

— Une valeur élevée de C pénalise fortement les erreurs de classification ;

— Une valeur faible de C autorise davantage d'erreurs mais peut améliorer la capacité de généralisation.

2.4.1.5 SVM linéaire et SVM non linéaire

Les SVM peuvent être divisés en deux grandes catégories : les SVM linéaires et les SVM non linéaires.

Le SVM linéaire est utilisé lorsque les données peuvent être séparées par un hyperplan dans l'espace des caractéristiques. Il est particulièrement adapté aux problèmes comportant un grand nombre de variables.

Lorsque les données ne sont pas linéairement séparables, il est possible d'utiliser un SVM non linéaire basé sur des fonctions noyaux (*Kernel Functions*). Ces fonctions permettent de projeter les données dans un espace de dimension supérieure où une séparation linéaire devient possible.

Les noyaux les plus utilisés sont :

— Le noyau linéaire (*Linear Kernel*) ;

— Le noyau polynomial (*Polynomial Kernel*) ;

— Le noyau gaussien (*Radial Basis Function - RBF*) ;

— Le noyau sigmoïde (*Sigmoid Kernel*).

Dans le cadre de la reconnaissance des chiffres manuscrits, le noyau linéaire offre généralement un bon compromis entre précision, simplicité et temps d'exécution.

2.4.1.6 Classification multi-classes : stratégie One-vs-Rest

Le SVM a été initialement conçu pour les problèmes de classification binaire. Cependant, la base de données MNIST contient dix classes correspondant aux chiffres de 0 à 9.

Pour résoudre ce problème multi-classes, la stratégie *One-vs-Rest* (OvR) est généralement utilisée. Cette approche consiste à construire un classifieur indépendant pour chaque classe.

Chaque classifieur produit un score :

$$f_k(x) = w_k^T x + b_k$$

où k représente la classe considérée.

Lors de la phase de prédiction, tous les scores sont calculés simultanément et la classe retenue correspond au score maximal :

$$\hat{y} = \arg\max_k f_k(x)$$

Cette stratégie permet d'étendre efficacement le SVM aux problèmes multi-classes tout en conservant les avantages de la méthode originale.

Grâce à sa robustesse théorique, sa capacité de généralisation et ses bonnes performances sur des données de grande dimension, le SVM demeure l'un des algorithmes de classification les plus utilisés dans les domaines de la reconnaissance de formes, de la vision par ordinateur et de l'apprentissage automatique.

2.4.2 Implémentation du SVM dans l'article

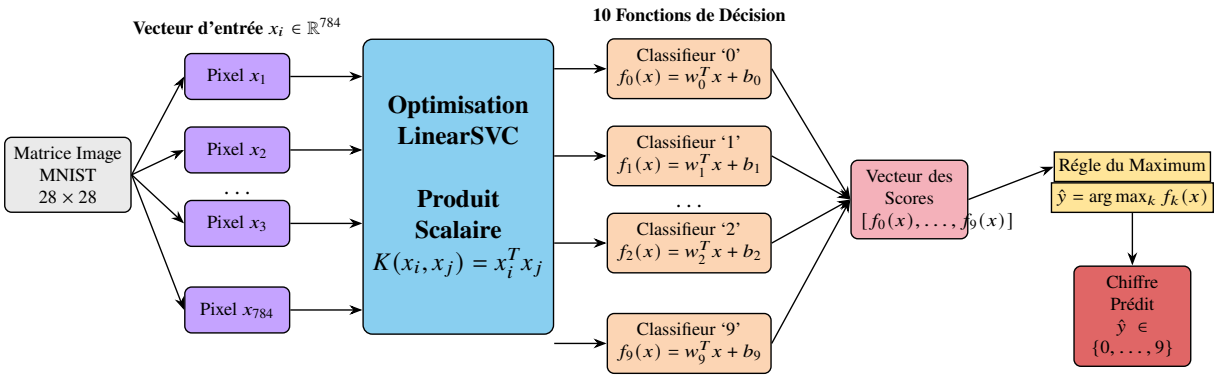


FIGURE 1 – Architecture fonctionnelle du classifieur SVM linéaire appliqué à la base MNIST.

Après le prétraitement des données, les auteurs ont utilisé l'algorithme Support Vector Machine (SVM) pour effectuer la classification des chiffres manuscrits de la base de données MNIST. L'implémentation a été réalisée à l'aide de la bibliothèque **Scikit-learn**, qui propose plusieurs variantes du SVM telles que *SVC*, *NuSVC* et *LinearSVC*. Dans cette étude, le modèle retenu est *LinearSVC*, qui utilise un noyau linéaire et repose sur la bibliothèque d'optimisation **LIBLINEAR**.

Le choix d'un noyau linéaire est particulièrement adapté aux données MNIST puisque chaque image est représentée par un grand nombre de caractéristiques. Le noyau linéaire calcule directement le produit scalaire entre deux observations :

$$K(x_i, x_j) = x_i^T x_j \tag{1}$$

où x_i et x_j désignent deux vecteurs de caractéristiques.

La base de données MNIST contient 60 000 images d'apprentissage et 10 000 images de test. Chaque image possède une résolution de 28×28 pixels. Après vectorisation, chaque image est représentée par :

$$x_i = [x_1, x_2, \ldots, x_{784}] \tag{2}$$

où les 784 caractéristiques correspondent aux intensités des pixels de l'image.

L'objectif du modèle consiste à construire un hyperplan séparateur capable de distinguer les différentes classes. Cet hyperplan est défini par :

$$w^T x + b = 0 \tag{3}$$

où w représente le vecteur des poids appris par le modèle et b le biais.

Pour chaque observation, le modèle calcule une fonction de décision :

$$f(x) = w^T x + b \tag{4}$$

Cette fonction indique la position de l'observation par rapport à la frontière de décision. Plus la valeur de $f(x)$ est élevée, plus le modèle est confiant dans sa prédiction.

Afin de déterminer l'hyperplan optimal, *LinearSVC* résout un problème d'optimisation basé sur la maximisation de la marge entre les classes. Dans la pratique, les données ne sont généralement pas parfaitement séparables. Une formulation à marge souple est donc utilisée. La fonction objectif minimisée est donnée par :

$$\min_{w,b} \frac{1}{2}\|w\|^2 + C \sum_{i=1}^{60000} \max(0, 1 - y_i(w^T x_i + b)) \tag{5}$$

où :

— w représente le vecteur des poids ;

— b représente le biais ;

— C est le paramètre de régularisation ;

— y_i est l'étiquette associée à l'échantillon i ;

— 60000 correspond au nombre total d'échantillons d'apprentissage de la base MNIST.

Cette fonction comporte deux composantes principales.

Le premier terme :

$$\frac{1}{2}\|w\|^2 \tag{6}$$

cherche à maximiser la marge de séparation en réduisant la norme du vecteur w.

Le second terme :

$$\sum_{i=1}^{60000} \max(0, 1 - y_i(w^T x_i + b)) \tag{7}$$

correspond à la fonction *Hinge Loss*, qui pénalise les observations mal classées ou situées trop près de la frontière de décision.

Le paramètre de régularisation C contrôle le compromis entre la maximisation de la marge et la réduction des erreurs de classification :

— une valeur élevée de C pénalise fortement les erreurs de classification ;

— une valeur faible de C favorise une marge plus large et améliore généralement la capacité de généralisation.

Durant l'apprentissage, l'algorithme ajuste progressivement les paramètres w et b afin de minimiser cette fonction coût.

Comme la base de données MNIST contient dix classes distinctes correspondant aux chiffres de 0 à 9, le problème est de nature multiclasses. Or, le SVM a été initialement conçu pour la classification binaire. Pour résoudre cette difficulté, la classe *LinearSVC* applique automatiquement la stratégie *One-vs-Rest (OvR)*.

Cette approche consiste à construire dix classifieurs binaires indépendants, de 0 a 9 . par exemple : un modèle pour reconnaître le chiffre 0 contre tous les autres

Pour chaque classe k, une fonction de décision est construite :

$$f_k(x) = w_k^T x + b_k \tag{8}$$

Lors de la phase de prédiction, les dix scores sont calculés simultanément :

$$f_0(x), f_1(x), f_2(x), \ldots, f_9(x) \tag{9}$$

La classe prédite correspond alors au score maximal :

$$\hat{y} = \arg \max_{k \in \{0,\ldots,9\}} f_k(x) \tag{10}$$

2.4.3 Multilayer Perceptron (MLP)

Le *Multilayer Perceptron* (MLP) est un modèle d'apprentissage supervisé appartenant à la famille des réseaux de neurones artificiels (*Artificial Neural Networks*). Il est conçu pour apprendre des relations complexes entre les données d'entrée et les sorties attendues à travers plusieurs couches de neurones interconnectés. Contrairement au perceptron simple, qui ne possède qu'une seule couche de sortie, le MLP comporte une ou plusieurs couches cachées lui permettant de résoudre des problèmes non linéaires. Cette capacité en fait l'un des modèles les plus utilisés dans les tâches de classification et de régression.

Le MLP appartient également à la catégorie des réseaux entièrement connectés (*Fully Connected Neural Networks*), où chaque neurone d'une couche est relié à tous les neurones de la couche précédente. En raison de cette structure, les couches entièrement connectées ne peuvent traiter que des vecteurs unidimensionnels en entrée. Lorsque les données sont représentées sous forme de matrices ou de tenseurs multidimensionnels, comme c'est souvent le cas pour les images, une opération appelée *Flattening* est généralement appliquée afin de transformer ces données en un vecteur unidimensionnel tout en conservant l'ensemble des valeurs originales.

Mathématiquement, si une donnée d'entrée est représentée par une matrice :

$$X \in \mathbb{R}^{m \times n} \tag{11}$$

l'opération de *Flatten* produit un vecteur :

$$X_f = [x_1, x_2, x_3, \ldots, x_{m \times n}] \tag{12}$$

Cette transformation ne modifie pas les valeurs contenues dans les données ; elle modifie uniquement leur organisation afin de les rendre compatibles avec les opérations matricielles effectuées par les couches *Dense* du réseau. Toutefois, le *Flattening* présente une limitation importante : il supprime les relations spatiales ou structurelles éventuellement présentes dans les données d'origine.

Une fois les données converties sous forme vectorielle, elles peuvent être propagées à travers les différentes couches du réseau afin d'extraire progressivement les caractéristiques les plus pertinentes pour la tâche d'apprentissage considérée.

2.4.3.1 Architecture générale du MLP

Un réseau MLP est généralement composé de trois types de couches :

— une couche d'entrée (*Input Layer*)

— une ou plusieurs couches cachées (*Hidden Layers*)

— une couche de sortie (*Output Layer*)

Chaque neurone d'une couche est connecté à tous les neurones de la couche suivante à travers des poids synaptiques.

L'architecture générale peut être représentée par :

$$Input \rightarrow Hidden_1 \rightarrow Hidden_2 \rightarrow \cdots \rightarrow Output \tag{13}$$

La couche d'entrée reçoit les caractéristiques du problème tandis que la couche de sortie produit la prédiction finale du réseau.

Les couches cachées réalisent les transformations intermédiaires permettant d'extraire progressivement des représentations pertinentes des données.

2.4.3.2 Modèle mathématique d'un neurone

Le neurone artificiel constitue l'élément fondamental du réseau.

Pour un neurone recevant n entrées, la combinaison linéaire est calculée par :

$$z = \sum_{i=1}^{n} w_i x_i + b \tag{14}$$

où :

— x_i représente la i^{me} entrée

— w_i représente le poids associé

— b représente le biais

— z représente la somme pondérée.

La sortie du neurone est ensuite obtenue par l'application d'une fonction d'activation :

$$a = f(z) \tag{15}$$

où f désigne la fonction d'activation choisie.

2.4.3.3 Propagation avant (Forward Propagation)

La propagation avant constitue la phase durant laquelle l'information circule de la couche d'entrée vers la couche de sortie.

Pour une couche l, le calcul s'effectue selon :

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)} \tag{16}$$

où :

— $W^{(l)}$ est la matrice des poids

— $a^{(l-1)}$ représente les sorties de la couche précédente

— $b^{(l)}$ représente le vecteur des biais.

La sortie de la couche est ensuite obtenue par :

$$a^{(l)} = f(z^{(l)}) \tag{17}$$

Ce processus est répété jusqu'à la couche de sortie où le réseau produit sa prédiction finale.

2.4.3.4 Fonctions d'activation

Les fonctions d'activation introduisent la non-linéarité nécessaire à l'apprentissage de relations complexes.

Fonction Sigmoïde La fonction sigmoïde est définie par :

$$f(x) = \frac{1}{1 + e^{-x}} \tag{18}$$

Sa sortie appartient à l'intervalle :

$$0 \leq f(x) \leq 1 \tag{19}$$

Elle est principalement utilisée dans les problèmes de classification binaire.

Fonction ReLU La fonction ReLU (*Rectified Linear Unit*) est actuellement la plus utilisée dans les couches cachées.

Elle est définie par :

$$f(x) = \max(0, x) \tag{20}$$

Cette fonction présente plusieurs avantages :

— simplicité de calcul

— réduction du problème du gradient évanescent

— accélération de l'apprentissage.

Fonction Softmax Pour les problèmes de classification multi-classes, la couche de sortie utilise généralement la fonction Softmax.

Pour une sortie composée de K classes :

$$P(y = k) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}} \tag{21}$$

Cette fonction transforme les sorties du réseau en probabilités dont la somme vaut :

$$\sum_{k=1}^{K} P(y = k) = 1 \tag{22}$$

La classe prédite est alors :

$$\hat{y} = \arg\max_{k} P(y = k) \tag{23}$$

2.4.3.5 Fonction de coût

La qualité des prédictions du réseau est mesurée à l'aide d'une fonction de coût.

Dans les problèmes de classification, la fonction d'entropie croisée (*Cross-Entropy Loss*) est largement utilisée :

$$J(\theta) = -\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik}\log(\hat{y}_{ik}) \tag{24}$$

où :

— N représente le nombre d'exemples

— K représente le nombre de classes

— y_{ik} représente la valeur réelle

— $\hat{y}_{ik}$ représente la probabilité prédite.

L'objectif du réseau consiste à minimiser cette fonction.

2.4.3.6 Rétropropagation du gradient (Backpropagation)

L'apprentissage du MLP repose sur l'algorithme de rétropropagation (*Backpropagation*).

Après le calcul de la fonction de coût, l'erreur est propagée depuis la couche de sortie vers les couches précédentes afin de déterminer l'influence de chaque poids sur l'erreur globale.

Cette étape repose sur le calcul du gradient :

$$\frac{\partial J}{\partial w} \tag{25}$$

Les poids sont ensuite mis à jour selon la règle de descente de gradient :

$$w_{new} = w_{old} - \eta\frac{\partial J}{\partial w} \tag{26}$$

où η représente le taux d'apprentissage (*learning rate*).

Cette opération est répétée jusqu'à ce que le modèle converge vers une solution satisfaisante.

2.4.4 Implémentation du MLP dans l'article

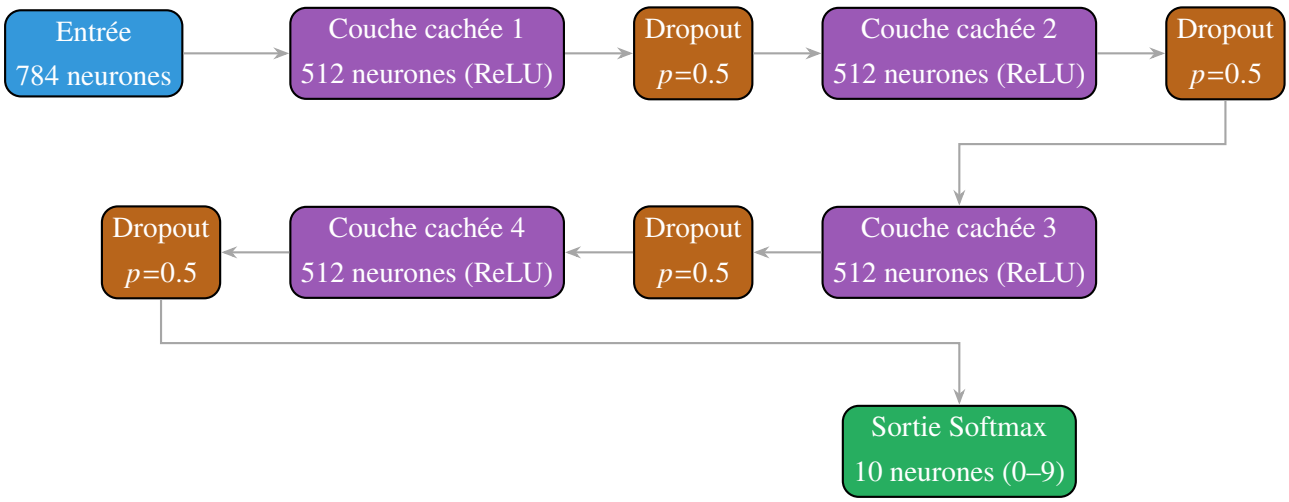


FIGURE 2 – Architecture du modèle MLP pour la reconnaissance de chiffres manuscrits (MNIST)
.

Après le prétraitement des données, les auteurs ont mis en œuvre un modèle de type *Multilayer Perceptron* (MLP) à l'aide de la bibliothèque Keras. Le réseau est construit selon une architecture séquentielle (*Sequential Model*), dans laquelle les couches sont ajoutées successivement afin de former un réseau de neurones entièrement connecté (*Fully Connected Neural Network*).

Les images du dataset MNIST possèdent une taille de 28×28 pixels. Avant leur introduction dans le réseau, elles sont transformées en vecteurs unidimensionnels de :

$$28 \times 28 = 784$$

caractéristiques. Chaque neurone de la couche d'entrée reçoit donc l'une des 784 valeurs représentant les intensités des pixels de l'image.

L'architecture proposée dans l'article est composée de quatre couches cachées ainsi que d'une couche de sortie. Chaque couche cachée contient :

$$512$$

neurones. Le choix d'un nombre élevé de neurones permet au réseau d'apprendre des représentations complexes des chiffres manuscrits et d'extraire progressivement les caractéristiques discriminantes nécessaires à la classification.

2.4.4.1 Propagation dans les couches cachées

Pour une couche donnée, la sortie d'un neurone est calculée à partir de la combinaison linéaire des entrées :

$$z = \sum_{i=1}^{n} w_i x_i + b$$

où :

— x_i représente les entrées du neurone ;

— w_i représente les poids associés aux connexions ;

— b représente le biais.

La valeur obtenue est ensuite transformée à l'aide d'une fonction d'activation.

2.4.4.2 Fonction d'activation ReLU

Afin d'introduire une non-linéarité dans le réseau, les auteurs ont utilisé la fonction d'activation ReLU (*Rectified Linear Unit*) dans les couches cachées.

La sortie d'un neurone est définie par :

$$ReLU(z) = \max(0, z)$$

Cette fonction permet d'éviter certains problèmes liés aux fonctions d'activation classiques, notamment la disparition du gradient (*Vanishing Gradient*), tout en accélérant le processus d'apprentissage. Elle contribue également à l'apprentissage de frontières de décision non linéaires plus complexes.

2.4.4.3 Régularisation par Dropout

Afin de réduire le phénomène d'overfitting, les auteurs ont ajouté des couches Dropout entre les couches cachées.

Le principe consiste à désactiver aléatoirement une proportion p de neurones pendant l'entraînement :

$$A_{drop} = M \odot A$$

où :

— A représente le vecteur des activations de la couche ;

— M est un masque aléatoire constitué de valeurs 0 et 1 ;

— $\odot$ représente le produit élément par élément.

Pour chaque neurone :

$$M_i = \begin{cases} 0 & \text{avec probabilité } p \\ 1 & \text{avec probabilité } 1 - p \end{cases}$$

Si :

$$p = 0.5$$

alors environ 50% des neurones sont temporairement désactivés à chaque itération d'entraînement.

Cette technique empêche le réseau de devenir excessivement dépendant de certains neurones particuliers. Elle force ainsi les différentes couches à apprendre des représentations plus robustes et améliore la capacité de généralisation du modèle sur des données jamais observées auparavant.

2.4.4.4 Couche de sortie Softmax

La dernière couche du réseau est une couche Softmax composée de dix neurones correspondant aux dix classes du dataset MNIST :

$$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

La fonction Softmax transforme les sorties brutes du réseau en probabilités selon :

$$P(y = k) = \frac{e^{z_k}}{\sum_{j=1}^{10} e^{z_j}}$$

où :

— z_k représente la sortie du neurone associé à la classe k ;

— $P(y = k)$ représente la probabilité que l'image appartienne à la classe k.

La somme des probabilités est égale à :

$$\sum_{k=1}^{10} P(y = k) = 1$$

La classe prédite est obtenue en sélectionnant la probabilité la plus élevée :

$$\hat{y} = \arg\max_k P(y = k)$$

Cette fonction est particulièrement adaptée aux problèmes de classification multi-classes tels que la reconnaissance des chiffres manuscrits.

2.4.4.5 Fonction de perte

L'apprentissage du réseau est réalisé en minimisant une fonction de perte de type *Categorical Cross-Entropy*, adaptée aux problèmes de classification multi-classes.

La fonction de perte est définie par :

$$J = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{10} y_{ik}\log(\hat{y}_{ik})$$

où :

— N représente le nombre d'échantillons d'apprentissage ;

— y_{ik} représente la valeur réelle ;

— $\hat{y}_{ik}$ représente la probabilité prédite par la fonction Softmax.

Cette fonction mesure l'écart entre les probabilités prédites et les étiquettes réelles. L'objectif du processus d'apprentissage est de minimiser cette erreur afin d'améliorer progressivement les performances du réseau.

2.4.4.6 Apprentissage par rétropropagation

L'apprentissage du réseau est réalisé à l'aide de l'algorithme de rétropropagation (*Backpropagation*), qui ajuste progressivement les poids et les biais afin de minimiser la fonction de perte.

Les paramètres du modèle sont mis à jour selon le principe de la descente de gradient :

$$w_{new} = w_{old} - \eta\frac{\partial J}{\partial w}$$

où :

— J représente la fonction de perte ;

— η représente le taux d'apprentissage.

Grâce à cette architecture composée de plusieurs couches cachées, de la fonction ReLU, des couches de Dropout et de la fonction Softmax en sortie, le modèle MLP est capable d'apprendre des relations non linéaires complexes entre les pixels des images et les classes correspondantes.

2.4.5 Convolutional Neural Network (CNN)

Le Convolutional Neural Network (CNN) est une architecture avancée de réseaux de neurones artificiels spécialement conçue pour le traitement des données structurées sous forme de grilles, notamment les images numériques. Contrairement au Multilayer Perceptron (MLP) classique qui transforme directement l'image en un vecteur unidimensionnel, le CNN conserve la structure spatiale des images afin d'extraire automatiquement les caractéristiques importantes telles que les contours, les textures, les formes et les motifs visuels.

Les CNN sont aujourd'hui largement utilisés dans plusieurs domaines de l'intelligence artificielle, notamment la vision par ordinateur, la reconnaissance faciale, la classification d'images, la détection d'objets, l'imagerie médicale et la reconnaissance des caractères manuscrits.

2.4.5.1 Principe général d'un CNN

L'objectif principal d'un CNN est d'apprendre automatiquement des représentations hiérarchiques des données visuelles à travers plusieurs couches successives.

L'architecture générale d'un CNN est composée de :

— une couche d'entrée ;

— une ou plusieurs couches de convolution ;

— des fonctions d'activation ;

— des couches de pooling ;

— des couches entièrement connectées (Fully Connected) ;

— une couche de sortie.

Le pipeline général d'un CNN peut être représenté comme suit :

$$\textbf{Image} \rightarrow \textbf{Convolution} \rightarrow \textbf{ReLU} \rightarrow \textbf{Pooling} \rightarrow \textbf{Fully Connected} \rightarrow \textbf{Softmax}$$

Chaque couche joue un rôle spécifique dans le processus d'apprentissage.

2.4.5.2 Représentation mathématique des images

Une image numérique en niveaux de gris est représentée par une matrice :

$$\mathbf{X} \in \mathbb{R}^{\mathbf{H} \times \mathbf{W}}$$

où :

— H représente la hauteur de l'image ;

— W représente la largeur de l'image.

Pour une image couleur RGB, la représentation devient :

$$\mathbf{X} \in \mathbb{R}^{\mathbf{H} \times \mathbf{W} \times \mathbf{3}}$$

car l'image possède trois canaux : Rouge, Vert et Bleu.

Chaque pixel possède une intensité généralement comprise entre :

$$\mathbf{0} \leq \mathbf{x} \leq \mathbf{255}$$

Avant l'entraînement, une normalisation est souvent appliquée :

$$\mathbf{x'} = \frac{\mathbf{x}}{\mathbf{255}}$$

Après normalisation :

$$\mathbf{0} \leq \mathbf{x'} \leq \mathbf{1}$$

Cette étape améliore la stabilité numérique et accélère l'apprentissage du réseau.

2.4.5.3 Couche de convolution

La couche de convolution constitue l'élément fondamental du CNN. Son rôle est d'extraire automatiquement les caractéristiques importantes présentes dans l'image.

Un filtre de convolution, appelé également noyau (kernel), parcourt l'image afin de produire une carte de caractéristiques appelée Feature Map.

Mathématiquement, la convolution est définie par :

$$\mathbf{S(i,j)} = \sum_{\mathbf{m}} \sum_{\mathbf{n}} \mathbf{X(i+m, j+n)} \times \mathbf{K(m,n)}$$

où :

— X représente l'image d'entrée ;

— K représente le filtre de convolution ;

— $S(i, j)$ représente la valeur obtenue dans la Feature Map.

Le filtre apprend automatiquement les motifs importants durant l'entraînement.
Par exemple :

— certains filtres détectent les contours ;

— d'autres détectent les textures ;

— certains apprennent des formes complexes.

2.4.5.4 Paramètres de convolution

Plusieurs paramètres influencent le fonctionnement de la convolution.

Taille du noyau Les tailles de filtres les plus utilisées sont :

$$\mathbf{3 \times 3},\ \mathbf{5 \times 5},\ \mathbf{7 \times 7}$$

Les petits noyaux permettent généralement une meilleure extraction des détails locaux.

Stride Le paramètre Stride contrôle le déplacement du filtre sur l'image.
la taille de la sortie diminue plus rapidement.

Padding Le Padding consiste à ajouter des bordures de zéros autour de l'image afin de préserver les dimensions spatiales.

La taille de sortie après convolution est donnée par :

$$\textbf{Output} = \frac{\textbf{W} - \textbf{F} + \textbf{2P}}{\textbf{S}} + \textbf{1}$$

où :

— W représente la taille de l'entrée ;

— F représente la taille du filtre ;

— P représente le padding ;

— S représente le stride.

2.4.5.5 Fonction d'activation ReLU

Après chaque convolution, une fonction d'activation non linéaire est appliquée.

La fonction la plus utilisée dans les CNN est la fonction ReLU (Rectified Linear Unit), définie par :

$$\textbf{ReLU}(\mathbf{x}) = \max(\mathbf{0}, \mathbf{x})$$

Cette fonction permet :

— d'introduire la non-linéarité ;

— d'accélérer l'apprentissage ;

— de réduire le problème du vanishing gradient.

2.4.5.6 Couche de Pooling

Après la convolution, une couche de Pooling est généralement appliquée afin de réduire la dimension spatiale des Feature Maps.

Cette opération permet :

— de réduire le nombre de paramètres ;

— d'accélérer les calculs ;

— de limiter le surapprentissage ;

— de conserver les caractéristiques importantes.

La méthode la plus utilisée est le Max Pooling :

$$\mathbf{P}(\mathbf{i}, \mathbf{j}) = \max(\mathbf{X})$$

Par exemple, un Max Pooling 2×2 conserve uniquement la valeur maximale dans chaque région.

2.4.5.7 Extraction hiérarchique des caractéristiques

L'un des principaux avantages des CNN réside dans leur capacité à apprendre des représentations hiérarchiques.

Les premières couches apprennent généralement :

— les contours ;

— les lignes ;

— les textures simples.

Les couches intermédiaires apprennent :

— les formes ;

— les motifs complexes ;

— certaines parties d'objets.

Les couches profondes apprennent des représentations beaucoup plus abstraites et complexes.

2.4.5.8 Transformation Flatten

Les couches de convolution produisent des matrices multidimensionnelles. Cependant, les couches Fully Connected nécessitent des vecteurs unidimensionnels.

Une opération de Flatten est donc appliquée :

$$\mathbf{H} \times \mathbf{W} \times \mathbf{D} \rightarrow \mathbf{N}$$

où :

— H représente la hauteur ;

— W représente la largeur ;

— D représente le nombre de canaux ;

— N représente le vecteur final.

Cette étape transforme les cartes de caractéristiques en un vecteur exploitable par les couches denses.

2.4.5.9 Couches Fully Connected

Les couches Fully Connected réalisent la phase finale de classification.

Chaque neurone est connecté à tous les neurones de la couche précédente.

Le calcul d'un neurone est donné par :

$$\mathbf{z} = \sum_{\mathbf{i=1}}^{\mathbf{n}} \mathbf{w_i x_i} + \mathbf{b}$$

où :

— x_i représente les entrées ;

— w_i représente les poids ;

— b représente le biais.

2.4.5.10 Fonction Softmax

Pour les problèmes de classification multi-classes, la couche de sortie utilise généralement la fonction Softmax.

Cette fonction transforme les scores du réseau en probabilités :

$$\mathbf{P(y = k)} = \frac{\mathbf{e^{z_k}}}{\sum_{\mathbf{j=1}}^{\mathbf{K}} \mathbf{e^{z_j}}}$$

où :

— K représente le nombre total de classes ;

— z_k représente le score de la classe k.

Les probabilités obtenues vérifient :

$$\sum_{\mathbf{k=1}}^{\mathbf{K}} \mathbf{P(y = k)} = \mathbf{1}$$

La classe prédite correspond à :

$$\mathbf{\hat{y}} = \arg\max_{\mathbf{k}} \mathbf{P(y = k)}$$

2.4.5.11 Fonction de perte

Afin de mesurer l'erreur entre les prédictions et les vraies classes, une fonction de perte est utilisée.

Pour la classification multi-classes, la fonction la plus utilisée est la Cross Entropy :

$$\mathbf{L} = -\sum_{\mathbf{i=1}}^{\mathbf{K}} \mathbf{y_i} \log(\mathbf{\hat{y}_i})$$

où :

— y_i représente la valeur réelle ;

— $\hat{y}_i$ représente la probabilité prédite.

L'objectif de l'apprentissage consiste à minimiser cette fonction.

2.4.5.12 Rétropropagation des erreurs

Après le calcul de la perte, le CNN utilise l'algorithme de Backpropagation afin de mettre à jour les poids du réseau.

Le gradient est calculé par :

$$\frac{\partial \mathbf{L}}{\partial \mathbf{W}}$$

Les poids sont ensuite mis à jour selon :

$$\mathbf{W_{new}} = \mathbf{W_{old}} - \eta \frac{\partial \mathbf{L}}{\partial \mathbf{W}}$$

où :

— η représente le taux d'apprentissage.

2.4.5.13 Avantages des CNN

Les CNN présentent plusieurs avantages importants :

— extraction automatique des caractéristiques ;

— excellente performance sur les images ;

— réduction du nombre de paramètres grâce au partage des poids ;

— meilleure conservation de la structure spatiale ;

— forte capacité de généralisation.

Grâce à ces propriétés, les CNN constituent aujourd'hui l'une des architectures les plus performantes dans les problèmes de vision par ordinateur.

2.4.6 Implémentation du CNN dans l'article

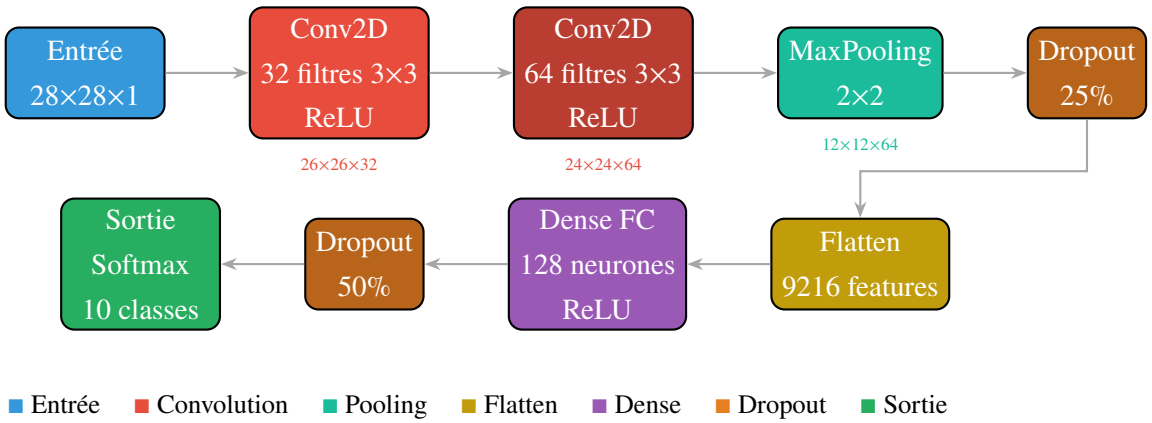


FIGURE 3 – Architecture du modèle CNN pour la reconnaissance de chiffres manuscrits (MNIST)

Les chercheurs ont utilisé un modèle de Deep Learning basé sur les Convolutional Neural Networks (CNN) afin de réaliser la reconnaissance des chiffres manuscrits du dataset MNIST. Cette implémentation a été réalisée à l'aide de la bibliothèque Keras, qui constitue une interface de haut niveau permettant de concevoir et d'entraîner efficacement des réseaux de neurones profonds.

Le modèle a été construit en utilisant la classe *Sequential*, qui permet d'ajouter les différentes couches du réseau de manière progressive, couche par couche. Cette approche facilite la mise en place de l'architecture et permet de structurer clairement le flux de traitement des données.

L'architecture proposée est composée de plusieurs types de couches, notamment des couches de convolution, de pooling, de régularisation Dropout ainsi que des couches entièrement connectées. Les données d'entrée traversent successivement ces différentes étapes de transformation, depuis l'extraction des caractéristiques jusqu'à la phase finale de classification, assurée par une couche de sortie utilisant la fonction Softmax.

2.4.6.1 Représentation des images d'entrée

Le dataset MNIST contient des images en niveaux de gris de taille :

$$\mathbf{28 \times 28}$$

Chaque image possède un seul canal :

$$\mathbf{C = 1}$$

Ainsi, la dimension complète de l'image est :

$$\mathbf{28 \times 28 \times 1}$$

Chaque pixel possède une intensité comprise entre :

$$\mathbf{0 \leq x \leq 255}$$

Avant l'entraînement, une normalisation est appliquée :

$$\mathbf{x' = \frac{x}{255}}$$

Après cette transformation :

$$\mathbf{0 \leq x' \leq 1}$$

Cette normalisation améliore la stabilité numérique du modèle, accélère la convergence et réduit les écarts de valeurs entre les pixels.

2.4.6.2 Première couche de convolution

La première couche convolutive utilise :

— 32 filtres ;

— une taille de filtre 3×3 ;

— un stride de 1 ;

— une fonction d'activation ReLU.

Un filtre est une petite matrice de poids qui parcourt l'image afin d'extraire certaines caractéristiques visuelles.

Un filtre de taille 3×3 peut être représenté par :

$$\mathbf{K} = \begin{bmatrix} w_1 & w_2 & w_3 \\ w_4 & w_5 & w_6 \\ w_7 & w_8 & w_9 \end{bmatrix}$$

L'opération de convolution est donnée par :

$$\mathbf{S(i,j)} = \sum_{\mathbf{m}} \sum_{\mathbf{n}} \mathbf{X(i+m, j+n) K(m,n)}$$

où :

— X représente l'image d'entrée ;

— K représente le filtre ;

— S représente la carte de caractéristiques produite.

Chaque filtre apprend automatiquement à détecter des motifs tels que les contours, les lignes ou les courbes présentes dans les chiffres manuscrits.

2.4.6.3 Dimension de sortie de la première convolution

La dimension de sortie d'une couche convolutive est calculée par :

$$\textbf{Output} = \frac{\textbf{N} + \textbf{2P} - \textbf{F}}{\textbf{S}} + \textbf{1}$$

où :

— N représente la taille de l'entrée ;

— P représente le padding ;

— F représente la taille du filtre ;

— S représente le stride.

Dans notre cas :

$$\textbf{N} = \textbf{28}, \quad \textbf{F} = \textbf{3}, \quad \textbf{P} = \textbf{0}, \quad \textbf{S} = \textbf{1}$$

Ainsi :

$$\textbf{Output} = \frac{\textbf{28} - \textbf{3}}{\textbf{1}} + \textbf{1} = \textbf{26}$$

La sortie de la première couche est donc :

$$\textbf{26} \times \textbf{26} \times \textbf{32}$$

Le nombre 32 correspond au nombre de filtres utilisés.

2.4.6.4 Fonction d'activation ReLU

Après la convolution, une fonction d'activation ReLU est appliquée :

$$\textbf{ReLU}(\textbf{x}) = \max(\textbf{0}, \textbf{x})$$

La fonction ReLU introduit une non-linéarité permettant au réseau d'apprendre des représentations complexes.

2.4.6.5 Deuxième couche de convolution

Une seconde couche convolutive est appliquée avec :

— 64 filtres ;

— une taille 3×3 ;

— un stride de 1 ;

— une activation ReLU.

La taille d'entrée est :

$$\mathbf{26 \times 26 \times 32}$$

La nouvelle dimension spatiale devient :

$$\frac{\mathbf{26 - 3}}{\mathbf{1}} + \mathbf{1} = \mathbf{24}$$

La sortie est donc :

$$\mathbf{24 \times 24 \times 64}$$

Cette couche apprend des caractéristiques plus complexes que celles extraites dans la première couche.

2.4.6.6 Couche de Max Pooling

Après les convolutions, une couche de Max Pooling est utilisée afin de réduire la taille des cartes de caractéristiques.

Les paramètres utilisés sont :

$$\mathbf{PoolSize} = (\mathbf{2}, \mathbf{2})$$

$$\mathbf{Stride} = \mathbf{2}$$

Le Max Pooling conserve uniquement la valeur maximale dans chaque région 2×2.

Par exemple :

$$\begin{bmatrix} 2 & 7 \\ 4 & 3 \end{bmatrix} \rightarrow \mathbf{7}$$

Après cette opération :

$$\mathbf{24 \times 24 \times 64 \rightarrow 12 \times 12 \times 64}$$

Cette réduction diminue le nombre de calculs tout en conservant les informations les plus importantes.

2.4.6.7 Régularisation par Dropout

Afin de réduire le phénomène de surapprentissage, une couche Dropout est appliquée avec :

$$\mathbf{Dropout} = \mathbf{0.25}$$

Mathématiquement :

$$\mathbf{m_i} \sim \mathbf{Bernoulli}(\mathbf{1 - p})$$

avec :

$$\mathbf{p = 0.25}$$

Les activations deviennent :

$$\mathbf{a'_i = m_i a_i}$$

Ainsi, environ 25% des neurones sont désactivés aléatoirement durant chaque itération d'entraînement.

2.4.6.8 Couche Flatten

Après le pooling, les cartes de caractéristiques possèdent la dimension :

$$\mathbf{12 \times 12 \times 64}$$

La couche Flatten transforme ces cartes en un vecteur unidimensionnel.
Le nombre total de caractéristiques devient :

$$\mathbf{12 \times 12 \times 64 = 9216}$$

La sortie est alors :

$$\mathbf{X} \in \mathbb{R}^{\mathbf{9216}}$$

Cette opération permet de connecter les couches convolutives aux couches Dense.

2.4.6.9 Couche entièrement connectée

Le vecteur obtenu est envoyé vers une couche Dense contenant :

$$\mathbf{128}$$

neurones.
Chaque neurone effectue le calcul suivant :

$$\mathbf{z = \sum_{i=1}^{9216} w_i x_i + b}$$

où :

— x_i représente les caractéristiques d'entrée ;

— w_i représente les poids ;

— b représente le biais.

Une activation ReLU est ensuite appliquée :

$$\mathbf{a} = \mathbf{ReLU}(\mathbf{z})$$

Cette couche combine les caractéristiques extraites afin de préparer la classification finale.

2.4.6.10 Deuxième couche Dropout

Une seconde couche Dropout est appliquée avec :

$$\mathbf{Dropout} = \mathbf{0.5}$$

Cela signifie que 50% des neurones sont désactivés aléatoirement durant l'entraînement. Cette technique réduit davantage le risque de surapprentissage.

2.4.6.11 Couche de sortie Softmax

La dernière couche contient :

$$\mathbf{10}$$

neurones correspondant aux chiffres :

$$\{\mathbf{0}, \mathbf{1}, \mathbf{2}, \mathbf{3}, \mathbf{4}, \mathbf{5}, \mathbf{6}, \mathbf{7}, \mathbf{8}, \mathbf{9}\}$$

La fonction Softmax transforme les scores en probabilités :

$$\mathbf{P}(\mathbf{y} = \mathbf{k}) = \frac{\mathbf{e}^{\mathbf{z_k}}}{\sum_{\mathbf{j=1}}^{\mathbf{10}} \mathbf{e}^{\mathbf{z_j}}}$$

avec :

$$\sum_{\mathbf{k=1}}^{\mathbf{10}} \mathbf{P}(\mathbf{y} = \mathbf{k}) = \mathbf{1}$$

La prédiction finale est donnée par :

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{k}} \mathbf{P}(\mathbf{y} = \mathbf{k})$$

Le chiffre associé à la probabilité la plus élevée est sélectionné comme résultat final.

2.5 Environnement expérimental

Dans le cadre de ce projet, plusieurs outils et bibliothèques ont été utilisés afin de réaliser les différentes expérimentations liées au Machine Learning et au Deep Learning. Ces technologies ont permis d'assurer le prétraitement des données, la conception des modèles, l'entraînement des réseaux de neurones ainsi que l'évaluation des performances obtenues. Les principaux outils utilisés sont présentés dans cette section.

2.5.1 Python

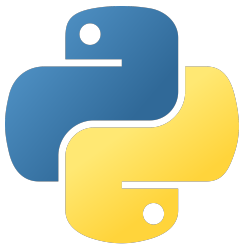


FIGURE 4 – Python

Python a été utilisé comme langage principal de programmation dans ce projet. Ce langage est particulièrement adapté aux applications de Machine Learning et de Deep Learning grâce à sa simplicité, sa flexibilité et à la richesse de ses bibliothèques scientifiques. Python permet également une implémentation rapide des algorithmes ainsi qu'une manipulation efficace des données numériques.

2.5.2 Google Colab



FIGURE 5 – Google Colab

Google Colab a été utilisé comme environnement de développement et d'exécution des expérimentations. Cette plateforme cloud permet d'exécuter des notebooks Python directement dans le navigateur sans installation locale complexe. Elle offre également l'accès à des ressources GPU permettant d'accélérer l'entraînement des modèles de Deep Learning tels que le MLP et le CNN.

2.5.3 NumPy



FIGURE 6 – NumPy

La bibliothèque NumPy a été utilisée pour le calcul scientifique et la manipulation des tableaux multidimensionnels. Elle permet d'effectuer efficacement les opérations mathématiques nécessaires au prétraitement des données, à la normalisation ainsi qu'aux différentes transformations numériques utilisées durant les expérimentations.

2.5.4 Scikit-Learn



FIGURE 7 – Scikit-Learn

Scikit-Learn a été utilisé pour l'implémentation des algorithmes de Machine Learning, notamment le Support Vector Machine (SVM). Cette bibliothèque fournit plusieurs outils performants pour le prétraitement des données, la réduction de dimension avec PCA, l'entraînement des modèles ainsi que l'évaluation des performances.

2.5.5 TensorFlow



FIGURE 8 – TensorFlow

TensorFlow a été utilisé comme framework principal de Deep Learning. Il permet la construction et l'optimisation des réseaux neuronaux grâce à des opérations mathématiques hautement optimisées. TensorFlow assure également le calcul automatique des gradients ainsi qu'une exécution efficace sur GPU.

2.5.6 Keras



FIGURE 9 – Keras

Keras, intégré à TensorFlow, a été utilisé pour construire les architectures des modèles MLP et CNN. Cette bibliothèque offre une interface simple et intuitive permettant de créer facilement des réseaux neuronaux profonds à travers différentes couches telles que Dense, Conv2D, MaxPooling, Flatten et Dropout.

2.6 Améliorations proposées

Afin d'améliorer les performances des modèles de classification étudiés dans ce travail, plusieurs techniques d'optimisation et de régularisation ont été intégrées durant les différentes phases d'implémentation. Ces améliorations concernent principalement le prétraitement des données, l'optimisation des hyperparamètres, la réduction du surapprentissage ainsi que l'amélioration de la capacité de généralisation des modèles. Chaque modèle a ainsi bénéficié de stratégies spécifiques permettant d'obtenir de meilleures performances en termes de précision, de stabilité d'apprentissage et de robustesse face aux nouvelles données.

2.6.1 Amélioration du SVM

Standardisation des données Après la normalisation des pixels dans l'intervalle $[0, 1]$, une étape supplémentaire de standardisation des données a été introduite dans cette étude à l'aide de la méthode *StandardScaler*. Contrairement à l'implémentation proposée dans l'article, qui se limitait uniquement à la normalisation, cette amélioration permet d'obtenir des données mieux adaptées au fonctionnement du SVM.

Le Support Vector Machine est un algorithme fortement basé sur les distances entre les observations. Ainsi, lorsque certaines caractéristiques possèdent des distributions différentes, elles peuvent influencer davantage la construction de l'hyperplan séparateur et dégrader les performances du modèle.

La standardisation consiste à centrer et réduire les données selon la formule :

$$z = \frac{x - \mu}{\sigma} \tag{27}$$

où :

— x représente la valeur originale ;

— μ représente la moyenne ;

— σ représente l'écart-type.

Après cette transformation :

$$\mu \approx 0 \tag{28}$$

et :

$$\sigma \approx 1 \tag{29}$$

Cette étape présente plusieurs avantages importants :

— équilibrer l'influence des différentes caractéristiques ;

— améliorer la stabilité numérique ;

— accélérer la convergence du modèle ;

— améliorer la précision du classifieur SVM.

L'ajout de cette étape constitue donc une amélioration importante par rapport à l'approche initiale proposée dans l'article.

Réduction de dimension avec PCA La base de données MNIST contient initialement 784 caractéristiques correspondant aux pixels des images. Un nombre élevé de caractéristiques augmente considérablement la complexité computationnelle du modèle et le temps d'entraînement.

Afin de résoudre ce problème, une réduction de dimension par *Principal Component Analysis* (PCA) a été utilisée. Le PCA permet de conserver uniquement les caractéristiques les plus importantes tout en supprimant les informations redondantes et le bruit présent dans les données.

Mathématiquement, le PCA repose sur la matrice de covariance :

$$C = \frac{1}{n} X^T X \tag{30}$$

Les valeurs propres et vecteurs propres sont ensuite calculés afin d'identifier les directions contenant le maximum d'information statistique.

Dans cette implémentation, le paramètre :

$$PCA(n_components = 0.95) \tag{31}$$

a été utilisé. Cela signifie que :

$$95\% \tag{32}$$

de la variance totale des données est conservée.

Le choix de cette valeur permet de réduire significativement le nombre de dimensions tout en préservant l'essentiel de l'information utile. La réduction obtenue est approximativement :

$$784 \rightarrow 331 \tag{33}$$

caractéristiques.

Cette étape a été ajoutée afin :

— de réduire le bruit ;

— d'accélérer le temps d'entraînement ;

— de diminuer la complexité du modèle ;

— d'améliorer la capacité de généralisation.

Ainsi, contrairement à l'article où aucune réduction de dimension avancée n'était utilisée, cette approche permet d'obtenir un modèle plus rapide et plus performant.

2.6.1.1 Principe mathématique du SVM

Le Support Vector Machine (SVM) est un algorithme supervisé utilisé pour les problèmes de classification. Son objectif principal consiste à déterminer un hyperplan optimal capable de séparer les différentes classes avec la plus grande marge possible.

L'équation générale de l'hyperplan est donnée par :

$$w \cdot x + b = 0 \tag{34}$$

où :

— w représente le vecteur des poids ;

— x représente les données ;

— b représente le biais.

Le SVM cherche à maximiser la marge entre les classes afin d'améliorer la généralisation du modèle sur les nouvelles données.

Le problème d'optimisation consiste à minimiser :

$$\min \frac{1}{2}||w||^2 \tag{35}$$

sous la contrainte :

$$y_i(w \cdot x_i + b) \geq 1 \tag{36}$$

Cette approche permet d'obtenir une frontière de décision robuste et performante.

Le choix du SVM dans cette étude s'explique par ses excellentes performances dans les problèmes de classification à haute dimension comme MNIST.

2.6.1.2 Utilisation du noyau RBF

Dans l'article, les chercheurs ont utilisé un noyau linéaire à travers la classe *LinearSVC*. Cependant, les chiffres manuscrits de MNIST ne sont pas parfaitement séparables de manière linéaire.

Pour cette raison, un noyau non linéaire de type *Radial Basis Function* (RBF) a été utilisé dans cette implémentation :

$$kernel = \texttt{'rbf'} \tag{37}$$

Le noyau RBF est défini mathématiquement par :

$$K(x_i, x_j) = e^{-\gamma||x_i - x_j||^2} \tag{38}$$

où :

— $||x_i - x_j||^2$ représente la distance euclidienne ;

— γ contrôle l'influence des voisins.

Le noyau RBF permet de projeter implicitement les données dans un espace de dimension supérieure afin de rendre les classes plus facilement séparables.

Le choix du noyau RBF permet :

— de capturer des relations non linéaires complexes ;

— d'améliorer la séparation des classes ;

— d'augmenter la précision de classification.

Cette modification représente l'une des principales améliorations apportées à l'architecture initiale de l'article.

2.6.1.3 Paramètre *C*

Le paramètre utilisé dans cette implémentation est :

$$C = 10 \tag{39}$$

Le paramètre C contrôle le compromis entre :

— la maximisation de la marge ;

— la réduction des erreurs de classification.

Une valeur faible de C autorise davantage d'erreurs mais améliore parfois la généralisation, tandis qu'une valeur élevée pénalise fortement les erreurs.

Après plusieurs expérimentations, la valeur :

$$C = 10 \tag{40}$$

a été retenue car elle permet d'obtenir un bon équilibre entre précision et capacité de généralisation.

2.6.1.4 Paramètre γ

Le paramètre utilisé est :

$$\texttt{gamma = 'scale'} \tag{41}$$

Cette configuration permet à Scikit-Learn de calculer automatiquement la valeur optimale de γ selon :

$$\gamma = \frac{1}{n_{\text{features}} \times Var(X)} \tag{42}$$

Le paramètre γ contrôle la forme de la frontière de décision :

— un petit γ produit une frontière plus lisse ;

— un grand γ crée une frontière plus complexe.

Le choix de `gamma='scale'` permet d'obtenir un meilleur équilibre entre complexité et généralisation sans nécessiter un réglage manuel difficile.

2.6.2 Amélioration du MLP

Comme dans la plupart des approches de classification du dataset MNIST, les images ont tout d'abord été normalisées afin de ramener les valeurs des pixels dans l'intervalle $[0, 1]$. Les opérations classiques de *reshape* des images ainsi que l'encodage *One-Hot* des étiquettes ont également été réalisées afin de rendre les données compatibles avec l'architecture du réseau de neurones et avec la fonction de perte utilisée lors de l'entraînement.

En complément de ces étapes standards, une stratégie de **Data Augmentation** a été introduite afin d'améliorer la capacité de généralisation du modèle et de réduire le risque de surapprentissage (*overfitting*). Cette technique consiste à générer dynamiquement de nouvelles images d'entraînement à partir des images originales en appliquant différentes transformations géométriques aléatoires.

Une rotation aléatoire comprise entre $-8°$ et $+8°$ est appliquée selon la matrice de rotation :

$$R(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \tag{43}$$

avec :

$$-8^\circ \leq \theta \leq 8^\circ \tag{44}$$

Cette transformation permet au modèle d'apprendre à reconnaître correctement un chiffre même lorsqu'il est légèrement incliné.

Un zoom aléatoire est également appliqué avec :

$$0.92 \leq s \leq 1.08 \tag{45}$$

selon la transformation :

$$x' = sx \tag{46}$$

Cette opération améliore la robustesse du modèle face aux variations de taille des chiffres manuscrits.

Des translations horizontales et verticales sont aussi réalisées avec :

$$\text{width_shift} = 0.08 \tag{47}$$

$$\text{height_shift} = 0.08 \tag{48}$$

Ces transformations permettent au réseau d'apprendre à reconnaître les chiffres indépendamment de leur position exacte dans l'image.

L'intégration de la Data Augmentation constitue ainsi une amélioration importante par rapport à une simple utilisation des données originales. Elle augmente artificiellement la diversité du dataset d'entraînement, améliore la capacité de généralisation du modèle et contribue à obtenir de meilleures performances sur les données de test.

2.6.2.1 Architecture du MLP

L'architecture proposée est composée de :

$$\mathbf{784 \rightarrow 512 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 10} \tag{49}$$

où :

— **784 neurones** correspondent aux pixels de l'image d'entrée après l'opération de Flatten ;

— **512 neurones** dans la première couche cachée ;

— **512 neurones** dans la deuxième couche cachée ;

— **256 neurones** dans la troisième couche cachée ;

— **128 neurones** dans la quatrième couche cachée ;

— **10 neurones** dans la couche de sortie, correspondant aux dix classes du dataset MNIST (0 à 9).

La diminution progressive du nombre de neurones agit comme un mécanisme de compression de l'information. Les premières couches apprennent des caractéristiques générales présentes dans les images manuscrites, tandis que les couches plus profondes extraient des représentations de plus en plus discriminantes permettant de distinguer les différentes classes.

Cette organisation présente plusieurs avantages. Elle permet de réduire progressivement le nombre de paramètres du modèle, de diminuer le risque de surapprentissage (*overfitting*) et d'améliorer la capacité de généralisation du réseau sur des données jamais observées durant l'entraînement.

Le choix de cette architecture résulte d'un compromis entre la capacité d'apprentissage du modèle et sa complexité computationnelle. Un nombre important de neurones dans les premières couches favorise l'extraction d'un grand nombre de caractéristiques, tandis que la réduction progressive de la taille des couches permet de conserver uniquement les informations les plus pertinentes pour la tâche de classification.

2.6.2.2 Couche Flatten

Les images du dataset MNIST sont représentées sous forme de matrices de taille :

$$\mathbf{28 \times 28 \times 1} \tag{50}$$

Cependant, les couches entièrement connectées (*Dense*) utilisées dans un réseau MLP ne peuvent pas traiter directement des données bidimensionnelles ou tridimensionnelles. Il est donc nécessaire de transformer chaque image en un vecteur unidimensionnel avant son introduction dans le réseau.

Cette transformation est réalisée grâce à la couche **Flatten** :

$$\mathbf{28 \times 28 \times 1 \rightarrow 784} \tag{51}$$

Après cette opération, chaque image est représentée par :

$$\mathbf{X} = [\mathbf{x_1}, \mathbf{x_2}, \mathbf{x_3}, \ldots, \mathbf{x_{784}}] \tag{52}$$

où chaque composante correspond à l'intensité normalisée d'un pixel.

Cette transformation ne modifie pas les valeurs des pixels et ne provoque aucune perte d'information. Son unique objectif est de rendre les données compatibles avec l'architecture du MLP.

2.6.2.3 Fonction d'activation ReLU

Toutes les couches cachées du réseau utilisent la fonction d'activation ReLU (*Rectified Linear Unit*), définie par :

$$\mathbf{ReLU}(\mathbf{x}) = \max(\mathbf{0}, \mathbf{x}) \tag{53}$$

Cette fonction remplace toutes les valeurs négatives par zéro tout en conservant les valeurs positives.

Après application de la fonction d'activation, la sortie d'une couche devient :

$$\mathbf{A^{(l)}} = \mathbf{ReLU}\left(\mathbf{Z^{(l)}}\right) \tag{54}$$

L'utilisation de ReLU est essentielle car elle introduit une non-linéarité dans le réseau. Sans fonction d'activation, l'empilement de plusieurs couches Dense resterait équivalent à une simple transformation linéaire, limitant fortement les capacités d'apprentissage du modèle.

De plus, la fonction ReLU présente un coût de calcul très faible puisqu'elle repose uniquement sur une opération de comparaison avec zéro. Elle contribue également à réduire le problème du *vanishing gradient* rencontré avec certaines fonctions d'activation classiques telles que la sigmoïde ou la tangente hyperbolique, ce qui permet d'accélérer l'apprentissage et d'améliorer les performances globales du réseau.

2.6.2.4 Batch Normalization

Après chaque couche Dense, une couche de *Batch Normalization* est appliquée afin de stabiliser les activations du réseau et d'améliorer le processus d'apprentissage.

Pour un mini-batch contenant m exemples, la moyenne est calculée par :

$$\mu_{\mathbf{B}} = \frac{\mathbf{1}}{\mathbf{m}} \sum_{\mathbf{i=1}}^{\mathbf{m}} \mathbf{x_i} \tag{55}$$

et la variance par :

$$\sigma_{\mathbf{B}}^{\mathbf{2}} = \frac{\mathbf{1}}{\mathbf{m}} \sum_{\mathbf{i=1}}^{\mathbf{m}} (\mathbf{x_i} - \mu_{\mathbf{B}})^{\mathbf{2}} \tag{56}$$

Les activations sont ensuite normalisées selon :

$$\hat{\mathbf{x}}_{\mathbf{i}} = \frac{\mathbf{x_i} - \mu_{\mathbf{B}}}{\sqrt{\sigma_{\mathbf{B}}^{\mathbf{2}} + \varepsilon}} \tag{57}$$

où ε représente une petite constante permettant d'éviter une division par zéro.

Après cette étape de normalisation, une transformation affine est appliquée :

$$\mathbf{y_i} = \gamma \hat{\mathbf{x}}_\mathbf{i} + \beta \tag{58}$$

où :

— γ est un paramètre d'échelle appris automatiquement ;

— β est un paramètre de translation appris automatiquement.

L'intégration de la Batch Normalization présente plusieurs avantages importants :

— stabilisation des activations du réseau ;

— réduction des variations internes entre les couches ;

— accélération de la convergence ;

— amélioration de la propagation des gradients ;

— meilleure robustesse du modèle.

Cette technique permet donc d'obtenir un apprentissage plus rapide, plus stable et une meilleure capacité de généralisation du réseau de neurones.

2.6.2.5 Régularisation par Dropout

Afin de réduire le risque de surapprentissage (*overfitting*), plusieurs couches de *Dropout* ont été introduites dans l'architecture du réseau.

Les taux de Dropout utilisés sont les suivants :

— **30%** après la première couche cachée ;

— **30%** après la deuxième couche cachée ;

— **25%** après la troisième couche cachée ;

— **20%** après la quatrième couche cachée.

Le principe du Dropout consiste à désactiver aléatoirement une partie des neurones durant chaque itération d'entraînement. Cette opération empêche le réseau de devenir excessivement dépendant de certains neurones particuliers.

Mathématiquement, l'activation d'un neurone après application du Dropout est donnée par :

$$\mathbf{a}'_\mathbf{i} = \mathbf{m_i}\, \mathbf{a_i} \tag{59}$$

où :

— a_i représente l'activation originale du neurone ;

— a'_i représente l'activation après Dropout ;

— m_i représente un masque aléatoire.

Le masque est généré selon une loi de Bernoulli :

$$\mathbf{m_i} \sim \mathbf{Bernoulli(1 - p)} \tag{60}$$

où p représente le taux de Dropout.

Par exemple, lorsque :

$$\mathbf{p = 0.30} \tag{61}$$

environ 30% des neurones sont temporairement désactivés à chaque itération d'apprentissage.

L'utilisation du Dropout présente plusieurs avantages :

— réduction du phénomène d'overfitting ;

— amélioration de la capacité de généralisation ;

— diminution de la dépendance entre les neurones ;

— apprentissage de représentations plus robustes ;

— meilleure performance sur les données de test.

Cette technique constitue l'un des mécanismes de régularisation les plus efficaces pour les réseaux de neurones profonds et contribue significativement à l'amélioration des performances du modèle.

2.6.2.6 Régularisation L2

Une régularisation **L2** est appliquée sur l'ensemble des couches Dense du réseau à l'aide du coefficient :

$$\lambda = \mathbf{10^{-4}} \tag{62}$$

Cette technique consiste à ajouter une pénalité sur les poids du réseau directement dans la fonction de coût afin de limiter la croissance excessive des paramètres durant l'apprentissage.

La nouvelle fonction de coût devient :

$$\mathbf{L_{total}} = \mathbf{L} + \lambda \sum_{\mathbf{i=1}}^{\mathbf{n}} \mathbf{w_i^2} \tag{63}$$

où :

— L représente la perte principale du modèle ;

— w_i représente les poids du réseau ;

— λ représente le coefficient de pénalisation.

Le principe de la régularisation L2 consiste à pénaliser les poids trop élevés afin d'éviter que le modèle devienne excessivement complexe.

Sans régularisation, certains poids peuvent prendre des valeurs très importantes, ce qui pousse le réseau à mémoriser les données d'entraînement au lieu d'apprendre des caractéristiques généralisables. Ce phénomène conduit généralement au surapprentissage (*overfitting*).

L'utilisation de la régularisation L2 présente plusieurs avantages importants :

— réduction du risque d'overfitting ;

— limitation de la complexité du modèle ;

— amélioration de la capacité de généralisation ;

— apprentissage de représentations plus stables et plus robustes.

Cette technique constitue ainsi un mécanisme de régularisation essentiel pour améliorer les performances globales du réseau de neurones sur les données de test.

2.6.2.7 Couche de sortie Softmax

La dernière couche du réseau contient **10 neurones** correspondant aux dix chiffres du dataset MNIST :

$$\{\mathbf{0}, \mathbf{1}, \mathbf{2}, \mathbf{3}, \mathbf{4}, \mathbf{5}, \mathbf{6}, \mathbf{7}, \mathbf{8}, \mathbf{9}\} \tag{64}$$

Chaque neurone produit un score de sortie :

$$\mathbf{z_1}, \mathbf{z_2}, \mathbf{z_3}, \ldots, \mathbf{z_{10}} \tag{65}$$

Ces scores sont ensuite convertis en probabilités grâce à la fonction **Softmax** :

$$\mathbf{P(y = k)} = \frac{\mathbf{e^{z_k}}}{\sum_{\mathbf{j=1}}^{\mathbf{10}} \mathbf{e^{z_j}}} \tag{66}$$

Cette fonction transforme les sorties brutes du réseau en probabilités comprises entre 0 et 1. Elle garantit que :

$$\mathbf{0} \leq \mathbf{P(y = k)} \leq \mathbf{1} \tag{67}$$

et :

$$\sum_{\mathbf{k=1}}^{\mathbf{10}} \mathbf{P(y = k)} = \mathbf{1} \tag{68}$$

Ainsi, la somme des probabilités associées aux différentes classes est toujours égale à 1.

Le chiffre prédit correspond à la classe possédant la probabilité la plus élevée :

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{k}} \mathbf{P(y = k)} \tag{69}$$

Par exemple, si le réseau produit les probabilités suivantes :

$$[\mathbf{0.01}, \mathbf{0.02}, \mathbf{0.03}, \mathbf{0.01}, \mathbf{0.02}, \mathbf{0.88}, \mathbf{0.01}, \mathbf{0.01}, \mathbf{0.00}, \mathbf{0.01}] \tag{70}$$

alors le chiffre prédit est :

$$\hat{\mathbf{y}} = \mathbf{5} \tag{71}$$

car cette classe possède la probabilité la plus élevée.

L'utilisation de la fonction Softmax est particulièrement adaptée aux problèmes de classification multi-classes comme la reconnaissance des chiffres manuscrits, car elle permet d'obtenir une interprétation probabiliste claire des prédictions du modèle.

2.6.2.8 Label Smoothing

Dans cette implémentation, une technique de régularisation appelée **Label Smoothing** a été introduite afin de réduire l'excès de confiance du modèle durant l'apprentissage.

Le coefficient utilisé est :

$$\alpha = \mathbf{0.05} \tag{72}$$

Au lieu d'utiliser des étiquettes One-Hot strictes contenant uniquement des valeurs 0 et 1, les probabilités sont légèrement lissées selon la formule :

$$\mathbf{y_{smooth}} = (\mathbf{1} - \alpha)\mathbf{y} + \frac{\alpha}{\mathbf{K}} \tag{73}$$

où :

— y représente l'étiquette originale ;

— α représente le coefficient de lissage ;

— K représente le nombre total de classes.

Dans notre cas :

$$\mathbf{K} = \mathbf{10} \tag{74}$$

Ainsi :

$$\frac{\alpha}{\mathbf{K}} = \frac{\mathbf{0.05}}{\mathbf{10}} = \mathbf{0.005} \tag{75}$$

Par exemple, une image appartenant à la classe 5 possède initialement la représentation One-Hot suivante :

$$[\mathbf{0}, \mathbf{0}, \mathbf{0}, \mathbf{0}, \mathbf{0}, \mathbf{1}, \mathbf{0}, \mathbf{0}, \mathbf{0}, \mathbf{0}] \tag{76}$$

Après application du Label Smoothing, cette représentation devient approximativement :

$$[\mathbf{0.005}, \mathbf{0.005}, \mathbf{0.005}, \mathbf{0.005}, \mathbf{0.005}, \mathbf{0.955}, \mathbf{0.005}, \mathbf{0.005}, \mathbf{0.005}, \mathbf{0.005}] \tag{77}$$

Cette technique présente plusieurs avantages importants :

— réduction de l'excès de confiance du modèle ;

— amélioration de la capacité de généralisation ;

— diminution du risque d'overfitting ;

— amélioration de la robustesse des prédictions.

Le Label Smoothing permet ainsi d'obtenir un apprentissage plus stable et de meilleures performances sur les données de test.

2.6.2.9 Fonction de perte : Categorical Cross Entropy

Le problème étudié correspond à une tâche de classification multi-classes comportant **10 catégories** correspondant aux chiffres manuscrits de 0 à 9.

Afin de mesurer l'écart entre les prédictions du réseau et les classes réelles, la fonction de perte utilisée est la **Categorical Cross Entropy**.

Cette fonction est définie par :

$$\mathbf{L} = -\sum_{\mathbf{i=1}}^{\mathbf{K}} \mathbf{y_i} \log(\hat{\mathbf{y}}_\mathbf{i}) \tag{78}$$

où :

— K représente le nombre total de classes ;

— y_i représente la valeur réelle ;

— $\hat{y}_i$ représente la probabilité prédite par la fonction Softmax.

Dans notre cas :

$$\mathbf{K = 10} \tag{79}$$

Cette fonction pénalise fortement les prédictions incorrectes et récompense les prédictions proches de la vérité terrain.

Lorsque la probabilité prédite pour la bonne classe est élevée, la perte devient faible. En revanche, lorsque le modèle attribue une faible probabilité à la bonne classe, la perte augmente fortement.

L'objectif principal de l'apprentissage consiste donc à minimiser cette fonction de coût afin d'améliorer progressivement les performances du réseau de neurones.

L'utilisation de la Categorical Cross Entropy est particulièrement adaptée aux problèmes de classification multi-classes associés à une couche de sortie Softmax.

2.6.2.10 Optimiseur Adam

L'optimisation des paramètres du réseau est réalisée à l'aide de l'algorithme **Adam** (*Adaptive Moment Estimation*).

Adam combine les avantages de deux méthodes d'optimisation largement utilisées :

— **Momentum** ;

— **RMSProp**.

Le premier moment, représentant une moyenne mobile des gradients, est calculé par :

$$\mathbf{m_t} = \beta_1 \mathbf{m_{t-1}} + (\mathbf{1} - \beta_1)\mathbf{g_t} \tag{80}$$

avec :

$$\beta_1 = \mathbf{0.9} \tag{81}$$

Le second moment, représentant une moyenne mobile des carrés des gradients, est donné par :

$$\mathbf{v_t} = \beta_2 \mathbf{v_{t-1}} + (\mathbf{1} - \beta_2)\mathbf{g_t^2} \tag{82}$$

avec :

$$\beta_2 = \mathbf{0.999} \tag{83}$$

Après correction du biais, les poids du réseau sont mis à jour selon :

$$\mathbf{w_{t+1}} = \mathbf{w_t} - \eta \frac{\hat{\mathbf{m}}_\mathbf{t}}{\sqrt{\hat{\mathbf{v}}_\mathbf{t}} + \varepsilon} \tag{84}$$

où :

— η représente le taux d'apprentissage ;

— $\hat{m}_t$ représente le premier moment corrigé ;

— $\hat{v}_t$ représente le second moment corrigé ;

— ε représente une constante de stabilité numérique.

L'utilisation de l'optimiseur Adam présente plusieurs avantages importants :

— convergence plus rapide ;

— meilleure stabilité de l'apprentissage ;

— adaptation automatique du taux d'apprentissage ;

— amélioration des performances du réseau.

Cette méthode d'optimisation est particulièrement efficace pour les réseaux de neurones profonds comportant un grand nombre de paramètres.

2.6.2.11 Early Stopping et ReduceLROnPlateau

Afin d'améliorer la stabilité de l'apprentissage et de limiter le risque de surapprentissage, deux mécanismes supplémentaires ont été utilisés :

— Early Stopping ;

— ReduceLROnPlateau.

Early Stopping Le mécanisme d'**Early Stopping** permet d'interrompre automatiquement l'entraînement lorsque les performances du modèle cessent de s'améliorer sur les données de validation.

Le critère surveillé est :

$$\textbf{Validation Accuracy} \tag{85}$$

Le paramètre utilisé est :

$$\textbf{patience} = \textbf{10} \tag{86}$$

Ainsi, si aucune amélioration de la précision de validation n'est observée pendant dix époques consécutives, l'entraînement est automatiquement arrêté.

Les meilleurs poids obtenus durant l'apprentissage sont ensuite restaurés grâce à :

$$\textbf{restore_best_weights} = \textbf{True} \tag{87}$$

Cette stratégie permet :

— d'éviter un apprentissage excessif ;

— de réduire le risque d'overfitting ;

— d'améliorer la capacité de généralisation du modèle.

ReduceLROnPlateau Le mécanisme **ReduceLROnPlateau** permet d'ajuster automatiquement le taux d'apprentissage lorsque la progression du modèle devient faible.

Le critère surveillé est :

$$\textbf{Validation Loss} \tag{88}$$

Les paramètres utilisés sont :

$$\textbf{factor} = \textbf{0.5} \tag{89}$$

et :

$$\textbf{patience} = \textbf{3} \tag{90}$$

Lorsque la perte de validation cesse de diminuer pendant trois époques consécutives, le taux d'apprentissage est automatiquement réduit selon :

$$\mathbf{LR_{new}} = \mathbf{0.5} \times \mathbf{LR_{old}} \tag{91}$$

Cette réduction progressive du taux d'apprentissage permet :

— un ajustement plus précis des poids ;

— une meilleure stabilité de convergence ;

— une amélioration des performances durant les dernières phases de l'apprentissage.

L'association de l'Early Stopping et du ReduceLROnPlateau contribue ainsi à rendre l'apprentissage du réseau plus robuste et plus efficace.

2.6.3 Amélioration du CNN

Dans cette partie, nous présentons l'architecture du modèle CNN implémenté ainsi que les différentes techniques utilisées pour améliorer ses performances.

Au préalable, et en complément des étapes classiques de prétraitement des données (telles que la normalisation et la transformation des données), une stratégie de Data Augmentation a été mise en œuvre. Cette technique consiste à générer artificiellement de nouvelles images à partir des données d'entraînement en appliquant des transformations géométriques telles que la rotation, le zoom et les translations horizontales et verticales. L'objectif principal est d'augmenter la diversité des données et de rendre le modèle plus robuste face aux variations possibles des chiffres manuscrits, ce qui permet de réduire le phénomène de surapprentissage (overfitting).

2.6.3.1 Augmentation artificielle des données

Afin d'améliorer la capacité de généralisation du modèle et de limiter le phénomène de surapprentissage, une stratégie de *Data Augmentation* a été appliquée aux images d'entraînement.

Les transformations utilisées sont les suivantes :

— Rotation aléatoire : 10°

— Zoom : 10%

— Translation horizontale : 10%

— Translation verticale : 10%

Ces opérations permettent de générer dynamiquement de nouvelles variations des chiffres manuscrits tout en conservant leur signification. Le modèle devient ainsi plus robuste aux petites variations de position, d'échelle et d'orientation pouvant apparaître dans les données réelles.

2.6.3.2 Architecture du réseau CNN proposé

L'architecture développée est composée de deux blocs convolutionnels suivis d'une partie entièrement connectée destinée à la classification finale.

La structure générale du réseau est donnée par :

$$28 \times 28 \times 1 \rightarrow Conv32 \rightarrow Conv32 \rightarrow MaxPool \rightarrow Conv64 \rightarrow Conv64 \rightarrow MaxPool \rightarrow Flatten \rightarrow Dense256 \rightarrow Dense10$$

Cette architecture permet d'extraire progressivement des caractéristiques visuelles de plus en plus complexes avant d'effectuer la classification finale.

2.6.3.3 Premier bloc convolutionnel

Le premier bloc est constitué de deux couches de convolution utilisant chacune :

— 32 filtres ;

— une taille de noyau 3×3 ;

— la fonction d'activation ReLU ;

— un remplissage (*padding*) de type *Same*.

La première couche reçoit directement les images d'entrée :

$$Input = (28, 28, 1)$$

et applique 32 filtres de convolution.

Chaque filtre parcourt l'image afin de détecter des motifs élémentaires tels que les contours, les lignes horizontales, les lignes verticales, les courbes ainsi que les transitions de luminosité.

La seconde couche convolutionnelle applique à nouveau 32 filtres de taille 3×3. Cette couche ne travaille plus directement sur l'image brute mais sur les cartes de caractéristiques produites par la première couche. Elle permet ainsi de combiner les informations précédemment extraites afin de détecter des structures plus complexes caractéristiques des chiffres manuscrits.

Après chaque convolution, une couche de *Batch Normalization* est appliquée afin de stabiliser les activations et d'accélérer l'apprentissage.

2.6.3.4 Première couche de Max Pooling

Après les deux convolutions, une opération de *Max Pooling* est réalisée avec :

$$pool_size = (2, 2)$$

Cette opération réduit les dimensions spatiales des cartes de caractéristiques :

$$28 \times 28 \rightarrow 14 \times 14$$

Le Max Pooling conserve uniquement la valeur maximale dans chaque région de taille 2×2.

Cette réduction présente plusieurs avantages :

— diminution du nombre de paramètres ;

— réduction du coût de calcul ;

— amélioration de la robustesse aux petites translations ;

— limitation du surapprentissage.

2.6.3.5 Première couche Dropout

Après le Max Pooling, une couche :

$$Dropout = 0.25$$

est appliquée.

Cela signifie qu'environ :

$$25\%$$

des neurones sont désactivés aléatoirement durant chaque itération d'entraînement.

Cette stratégie empêche le réseau de devenir trop dépendant de certains neurones particuliers et améliore ainsi sa capacité de généralisation.

2.6.3.6 Deuxième bloc convolutionnel

Le deuxième bloc reprend la même structure mais avec un nombre plus élevé de filtres.

La première convolution utilise :

— 64 filtres ;

— un noyau de taille 3×3 ;

— la fonction ReLU ;

— un padding de type *Same*.

L'augmentation du nombre de filtres permet d'extraire davantage de caractéristiques visuelles.

Alors que les premières couches détectent principalement des contours simples, les couches plus profondes apprennent progressivement des formes plus abstraites et plus discriminantes.

Une seconde couche de convolution identique est ensuite appliquée :

— 64 filtres ;

— noyau 3×3 ;

— activation ReLU.

Comme dans le premier bloc, chaque convolution est suivie d'une couche de *Batch Normalization*.

2.6.3.7 Deuxième couche de Max Pooling

Une seconde opération de Max Pooling est réalisée avec :

$$pool_size = (2, 2)$$

Les dimensions spatiales deviennent alors :

$$14 \times 14 \rightarrow 7 \times 7$$

Cette réduction permet de concentrer l'information la plus importante tout en diminuant encore le nombre de paramètres à traiter.

2.6.3.8 Deuxième couche Dropout

Après cette étape, une nouvelle couche :

$$Dropout = 0.25$$

est appliquée afin de renforcer la régularisation du réseau et de limiter davantage le risque d'overfitting.

2.6.3.9 Couche Flatten

Les couches Dense ne peuvent pas traiter directement des tenseurs tridimensionnels produits par les couches convolutionnelles.

Une opération de *Flatten* est donc appliquée.

Cette opération transforme les cartes de caractéristiques :

$$7 \times 7 \times 64 = 3136$$

Le résultat est un vecteur de dimension :

$$3136$$

qui pourra être utilisé comme entrée des couches entièrement connectées.

2.6.3.10 Couche Dense cachée

Après Flatten, une couche Dense composée de :

$$256 \textbf{ neurones}$$

est utilisée avec la fonction d'activation ReLU.

Cette couche joue le rôle de classificateur. Elle combine toutes les caractéristiques extraites par les couches convolutionnelles afin de construire une représentation globale de l'image.

Une couche Batch Normalization est ensuite appliquée afin de stabiliser les activations.

Puis une couche :

$$Dropout = 0.5$$

est utilisée.

Cela signifie que :

$$50\%$$

des neurones sont désactivés aléatoirement pendant l'entraînement.

Cette valeur élevée est généralement utilisée dans les couches Dense car elles contiennent un grand nombre de paramètres et sont particulièrement sensibles au surapprentissage.

2.6.3.11 Couche de sortie

La dernière couche contient :

$$10 \textbf{ neurones}$$

correspondant aux 10 chiffres du dataset :

$$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

La fonction d'activation utilisée est Softmax :

$$P(y = k) = \frac{e^{z_k}}{\sum_{j=1}^{10} e^{z_j}}$$

Cette fonction transforme les scores produits par le réseau en probabilités dont la somme est égale à 1.

La classe prédite correspond à celle possédant la probabilité maximale.

2.6.3.12 Fonction de perte

Le problème traité étant une classification multi-classes, la fonction de coût utilisée est la *Categorical Cross Entropy* :

$$L = -\sum_{i=1}^{10} y_i \log(\hat{y}_i)$$

Cette fonction mesure l'écart entre les probabilités prédites par le réseau et les étiquettes réelles.

2.6.3.13 Optimiseur Adam

L'optimisation des poids du réseau est réalisée à l'aide de l'algorithme Adam :

$$optimizer = Adam$$

Adam combine les avantages des méthodes Momentum et RMSProp afin d'adapter automatiquement le taux d'apprentissage de chaque paramètre.

La mise à jour des poids est donnée par :

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

où η représente le taux d'apprentissage, $\hat{m}_t$ le premier moment corrigé et $\hat{v}_t$ le second moment corrigé.

2.6.3.14 Early Stopping

Afin d'éviter le surapprentissage, une stratégie Early Stopping est utilisée :

$$patience = 10$$

Le critère surveillé est :

$$Validation\ Loss$$

Si aucune amélioration n'est observée pendant 10 époques consécutives, l'entraînement est automatiquement interrompu.

Les meilleurs poids obtenus sont ensuite restaurés.

2.6.3.15 Réduction adaptative du taux d'apprentissage

Une stratégie *ReduceLROnPlateau* est utilisée avec :

$$factor = 0.5$$

et

$$patience = 3$$

Lorsque la perte de validation cesse de s'améliorer pendant trois époques consécutives, le taux d'apprentissage est divisé par deux :

$$LR_{new} = 0.5 \times LR_{old}$$

Cette stratégie permet un ajustement plus précis des poids durant les dernières phases de l'apprentissage.

2.7 Conclusion

Dans ce chapitre, la méthodologie adoptée pour la reconnaissance des chiffres manuscrits a été présentée, depuis les étapes de prétraitement des données jusqu'à l'implémentation des modèles SVM, MLP et CNN. Les approches proposées dans l'article de référence ont été reproduites puis améliorées à l'aide de différentes techniques d'optimisation et de régularisation.

L'ensemble des méthodes appliquées, notamment la normalisation des données, la Data Augmentation, le Dropout, la Batch Normalization et l'optimisation des hyperparamètres, a permis d'améliorer les performances de classification des modèles étudiés. Cette méthodologie constitue ainsi une base solide pour l'évaluation comparative des résultats qui sera présentée dans le chapitre suivant.

Chapitre 3 : Méthodologie d'évaluation

3.1 Introduction

Ce chapitre présente la méthodologie d'évaluation adoptée pour analyser les performances des modèles de reconnaissance des chiffres manuscrits. L'objectif est de mesurer et comparer la qualité des prédictions obtenues par les différents modèles implémentés, à savoir le SVM, le MLP et le CNN.

Pour cela, plusieurs métriques d'évaluation sont utilisées afin d'estimer la précision des modèles ainsi que leur capacité de généralisation sur des données inconnues. Les résultats obtenus sont d'abord présentés pour les implémentations initiales inspirées de l'article, puis comparés aux résultats obtenus après les différentes améliorations apportées.

3.2 Métriques d'évaluation

Afin d'évaluer et de comparer les performances des différents modèles de classification étudiés, deux métriques principales ont été utilisées : l'accuracy et le temps d'exécution. Ces métriques permettent d'analyser à la fois la qualité des prédictions produites par les modèles ainsi que leur coût computationnel.

3.2.1 Accuracy

L'accuracy représente la proportion d'exemples correctement classifiés parmi l'ensemble des observations évaluées. Elle constitue l'une des métriques les plus utilisées pour les problèmes de classification lorsque les classes sont relativement équilibrées.

Mathématiquement, l'accuracy est définie par :

$$\textbf{Accuracy} = \frac{\textbf{TP} + \textbf{TN}}{\textbf{TP} + \textbf{TN} + \textbf{FP} + \textbf{FN}} \tag{92}$$

où :

— **TP (True Positives)** représente le nombre de prédictions correctes positives ;

— **TN (True Negatives)** représente le nombre de prédictions correctes négatives ;

— **FP (False Positives)** représente le nombre de faux positifs ;

— **FN (False Negatives)** représente le nombre de faux négatifs.

Une valeur élevée de l'accuracy indique que le modèle est capable de prédire correctement la majorité des exemples du dataset. Dans cette étude, l'accuracy est calculée aussi bien sur l'ensemble d'entraînement que sur l'ensemble de test afin d'évaluer la capacité de généralisation des modèles.

3.2.2 Temps d'exécution

En complément de l'accuracy, le temps d'exécution a également été utilisé pour comparer les modèles. Cette métrique mesure la durée nécessaire à l'entraînement du modèle jusqu'à l'obtention des résultats finaux.

L'évaluation du temps d'exécution est particulièrement importante dans les applications réelles, car un modèle très précis peut nécessiter des ressources computationnelles importantes et un temps d'apprentissage élevé.

3.3 Résultats des modèles implémentés selon l'article

Modèle	Accuracy Entraînement (%)	Accuracy Test (%)	Temps d'exécution (min)
SVM	99.98	94.005	1.35
MLP	99.92	98.45	2.32
CNN	99.53	99.31	44.02

TABLE 1 – Comparaison des performances des modèles implémentés selon l'article

L'analyse des résultats montre que le modèle SVM obtient la meilleure précision sur les données d'entraînement avec un taux de 99.98%. Cependant, sa précision sur les données de test chute à 94.06%, ce qui suggère une capacité de généralisation plus limitée par rapport aux réseaux de neurones.

Le modèle MLP présente des performances plus équilibrées avec une précision de 99.59% sur l'ensemble d'entraînement et 98.45% sur l'ensemble de test. Cette faible différence entre les deux valeurs indique une bonne généralisation du modèle.

Le meilleur résultat sur les données de test est obtenu par le modèle CNN avec une précision de 99.31%. Cette performance confirme la capacité des réseaux convolutifs à extraire automatiquement les caractéristiques pertinentes des images et à réaliser une classification plus efficace des chiffres manuscrits.

Concernant le temps d'exécution, le SVM est le modèle le plus rapide avec seulement 1.35 minute d'entraînement. Le MLP nécessite 2.53 minutes, tandis que le CNN présente le temps d'exécution le plus élevé avec 44.20 minutes. Cette différence s'explique par la complexité de l'architecture convolutive et le nombre important d'opérations réalisées durant l'apprentissage.

Ainsi, même si le CNN est le modèle le plus coûteux en temps de calcul, il offre également les meilleures performances de classification sur le dataset MNIST.

3.3.1 Évolution de l'accuracy du modèle CNN

La Figure 13 présente l'évolution de la précision du modèle CNN au cours de l'apprentissage.

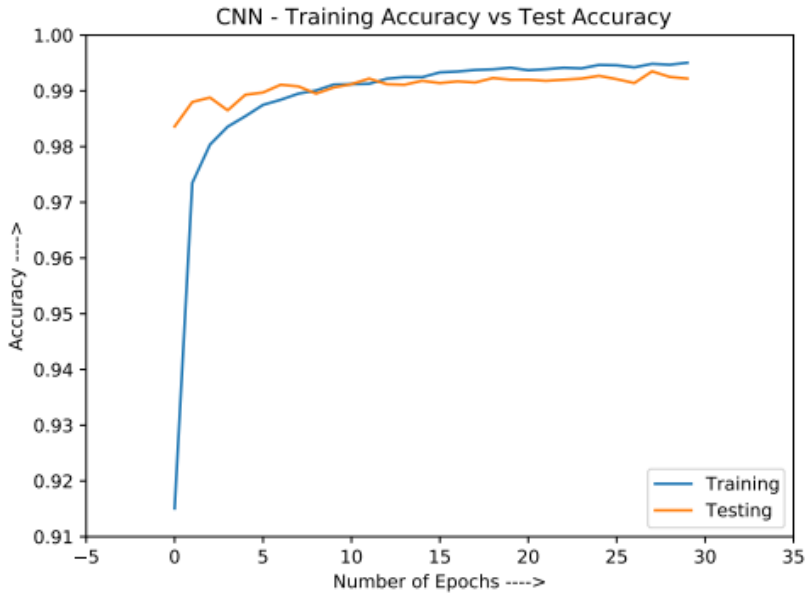


FIGURE 10 – Évolution de l'accuracy d'entraînement et de test du modèle CNN

La courbe montre une augmentation rapide de la précision durant les premières époques. L'accuracy d'entraînement passe d'environ 91% à plus de 98% après seulement quelques itérations. Parallèlement, l'accuracy de test suit une évolution similaire, ce qui indique que le modèle apprend efficacement les caractéristiques des images sans présenter de surapprentissage important.

À partir d'environ dix époques, les deux courbes deviennent relativement stables et convergent vers une valeur proche de 99%. Cette stabilisation indique que le réseau a atteint un niveau de performance optimal et que l'ajout d'époques supplémentaires apporte peu d'amélioration.

L'écart très faible observé entre les courbes d'entraînement et de test constitue également un indicateur de bonne généralisation. Le modèle conserve ainsi des performances élevées non seulement sur les données utilisées pour l'apprentissage, mais également sur les données jamais vues auparavant.

3.4 Résultats après amélioration des modèles

Le tableau suivant présente une comparaison des performances des modèles implémentés après amélioration. Les résultats montrent une amélioration significative des précisions obtenues par les trois modèles par rapport à ceux présentés dans l'article original.

Modèle	Accuracy Entraînement (%)	Accuracy Test (%)	Temps d'exécution (min)
SVM		97.44	30.00
MLP	97.28	99.19	20.00
CNN	99.44	99.64	15.00

TABLE 2 – Comparaison des performances des modèles implémentés

Le modèle CNN obtient les meilleures performances avec une précision de test de 99.64 %. Cette amélioration s'explique principalement par l'ajout de plusieurs techniques d'optimisation et de régularisation telles que la Data Augmentation, le Dropout, l'Early Stopping et la réduction adaptative du Learning Rate. De plus, les couches de convolution permettent au réseau d'extraire automatiquement les caractéristiques importantes des images, comme les contours et les formes des chiffres manuscrits, ce qui améliore fortement la capacité de classification.

Le modèle MLP atteint également une très bonne précision de 99.19 %. L'amélioration des performances est due à l'utilisation de plusieurs techniques modernes telles que la Batch Normalization, la régularisation L2, le Label Smoothing, le Dropout ainsi que la Data Augmentation. Ces techniques ont permis de réduire le phénomène de surapprentissage, de stabiliser l'apprentissage et d'améliorer la généralisation du modèle sur les données de test.

Concernant le modèle SVM, la précision obtenue est de 97.44 %, ce qui représente également une amélioration importante par rapport à l'article étudié. Cette amélioration est principalement liée à l'application de la normalisation des données ainsi qu'à l'utilisation de l'algorithme PCA, qui a permis de réduire la dimension des données tout en conservant les caractéristiques les plus importantes des images. Cela a facilité le travail du classificateur SVM et amélioré sa capacité de séparation entre les différentes classes.

Concernant le temps d'exécution, on remarque que celui du CNN a fortement diminué par rapport à l'article original. Cette réduction est principalement due à l'utilisation de techniques d'optimisation comme EarlyStopping et ReduceLROnPlateau. En effet, l'entraînement du modèle s'est arrêté automatiquement après convergence sans atteindre le nombre maximal d'époques, ce qui a permis de réduire considérablement le temps d'apprentissage tout en conservant une excellente précision.

En revanche, le temps d'exécution du MLP a légèrement augmenté. Cela s'explique par l'ajout de plusieurs couches cachées ainsi que par l'intégration de techniques supplémentaires comme la Batch Normalization, le Dropout, la régularisation L2 et la Data Augmentation, qui augmentent le nombre de calculs effectués durant l'entraînement.

Pour le modèle SVM, le temps d'exécution a également augmenté en raison de l'application de la réduction de dimension par PCA ainsi que du traitement supplémentaire effectué avant l'apprentissage. Même si PCA réduit la taille des données, son calcul initial ajoute un coût computationnel supplémentaire.

On remarque également que, dans certains cas, l'accuracy de test est légèrement supérieure à l'accuracy d'entraînement. Ce phénomène est normal dans les modèles utilisant des techniques de régularisation comme le Dropout et la Data Augmentation. Pendant l'entraînement, certaines données sont volontairement modifiées ou certains neurones sont désactivés aléatoirement afin de rendre l'apprentissage plus difficile et d'éviter le surapprentissage. En phase de test, ces mécanismes ne sont plus appliqués, ce qui permet au modèle d'obtenir parfois une meilleure précision sur les données de test que sur les données d'entraînement.

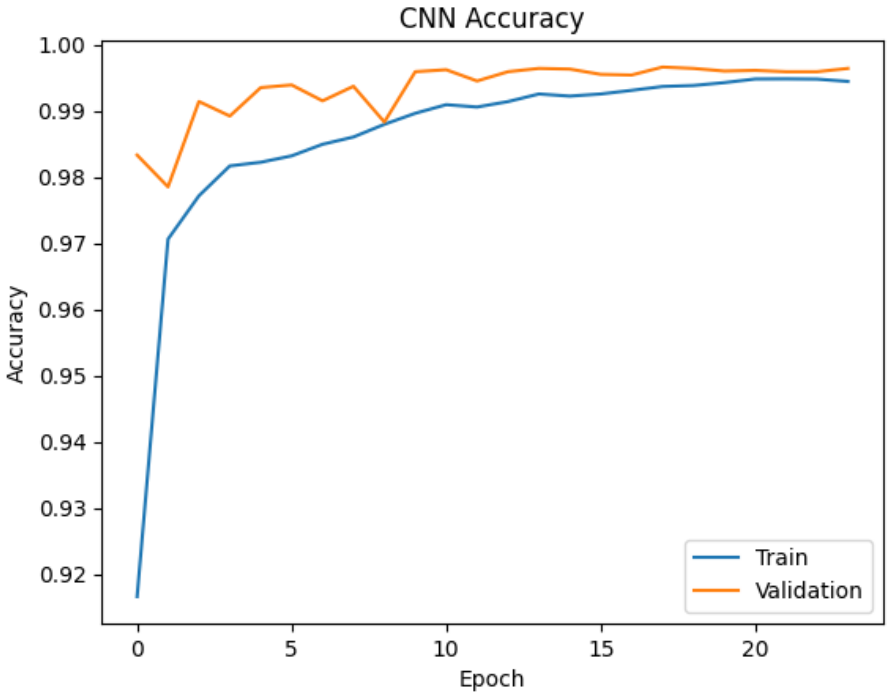


FIGURE 11 – Évolution de l'accuracy d'entraînement et de test du modèle CNN

La courbe d'accuracy du CNN montre une convergence rapide dès les premières époques, avec une stabilisation autour de 99.5% vers l'époque 15. La courbe de validation reste légèrement supérieure à celle d'entraînement, ce qui reflète l'effet du Dropout et de la Data Augmentation actifs uniquement pendant l'entraînement. L'absence de divergence entre les deux courbes confirme qu'il n'y a pas de surapprentissage. L'Early Stopping a automatiquement arrêté l'entraînement vers l'époque 23, au moment optimal.

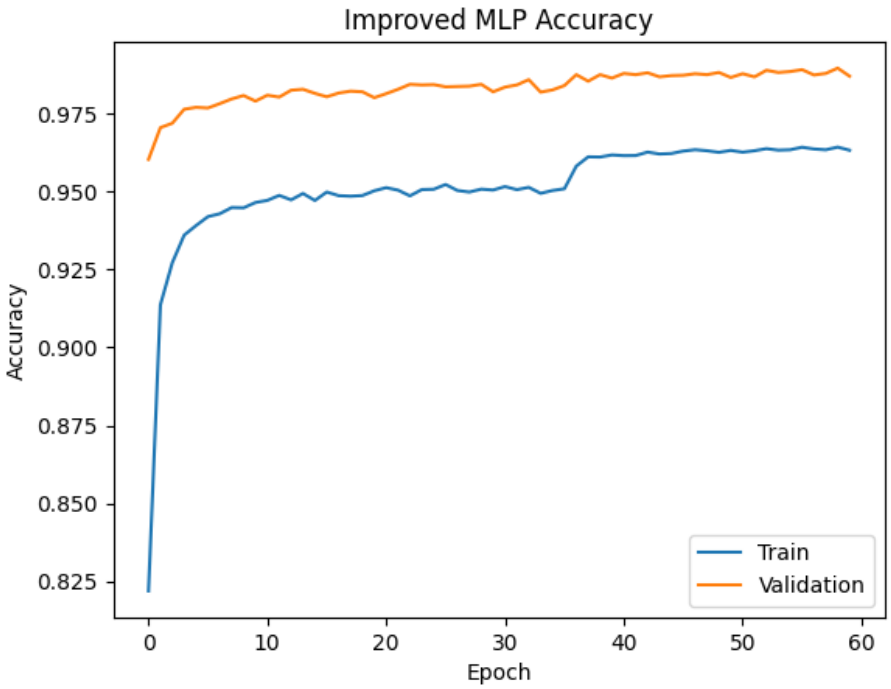


FIGURE 12 – Évolution de l'accuracy d'entraînement et de test du modèle MLP

La courbe d'accuracy du MLP montre une progression stable sur les 60 époques, avec une accuracy d'entraînement qui passe de 82% à environ 96%, tandis que la validation atteint près de 99%. L'écart entre les deux courbes s'explique par les techniques de régularisation (Dropout, Label Smoothing, L2) actives uniquement pendant l'entraînement. Cet écart reste stable tout au long de l'apprentissage, ce qui confirme une bonne généralisation sans surapprentissage.

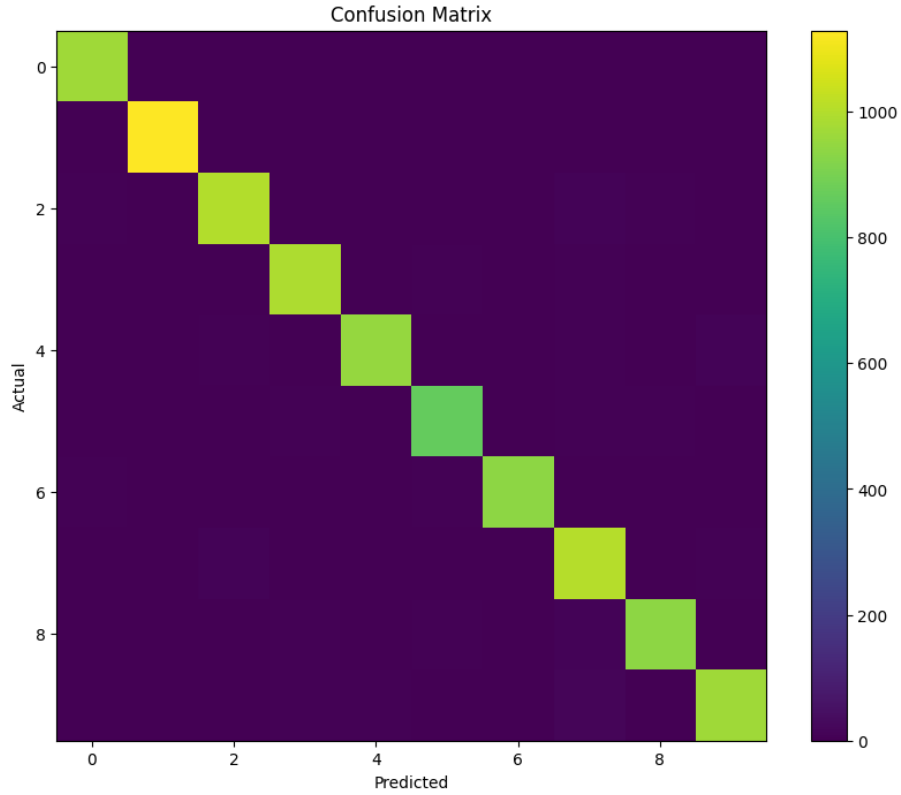


FIGURE 13 – MATRICE DE CONFUSION

La matrice de confusion du SVM montre une diagonale principale très dominante, confirmant l'accuracy globale de 97.44%. Les cases hors diagonale sont quasi nulles, indiquant très peu d'erreurs de classification. Aucune confusion systématique n'est observable entre les classes, ce qui témoigne de la robustesse du modèle sur l'ensemble des 10 chiffres.

3.5 Conclusion

Dans ce chapitre, une évaluation comparative des modèles SVM, MLP et CNN a été réalisée à l'aide de différentes métriques, notamment l'accuracy et le temps d'exécution. Les résultats obtenus montrent que les techniques de Deep Learning, en particulier le modèle CNN, offrent les meilleures performances pour la reconnaissance des chiffres manuscrits sur le dataset MNIST.

Les différentes améliorations apportées aux modèles, telles que la Data Augmentation, le Dropout, la Batch Normalization, la régularisation L2 et l'Early Stopping, ont permis d'améliorer significativement la précision des modèles tout en réduisant le phénomène de surapprentissage.

L'analyse des courbes d'apprentissage et des matrices de confusion confirme également la bonne capacité de généralisation des modèles implémentés. Ainsi, cette étude met en évidence l'efficacité des approches modernes de Deep Learning dans les tâches de classification d'images manuscrites.

Conclusion Générale

Ce travail avait pour objectif d'étudier et de comparer plusieurs approches d'apprentissage automatique et d'apprentissage profond pour la reconnaissance des chiffres manuscrits à partir du dataset MNIST. Dans un premier temps, les modèles SVM, MLP et CNN proposés dans l'article de référence ont été reproduits afin d'analyser leur fonctionnement ainsi que leurs performances de classification.

Par la suite, plusieurs améliorations ont été apportées aux différents modèles afin d'augmenter leur capacité de généralisation et de réduire le phénomène de surapprentissage. Parmi les principales techniques utilisées figurent la normalisation des données, la Data Augmentation, la Batch Normalization, le Dropout, la régularisation L2, le Label Smoothing, l'Early Stopping ainsi que l'optimisation adaptative du taux d'apprentissage.

Les résultats expérimentaux obtenus montrent que les modèles de Deep Learning, et plus particulièrement le CNN, offrent les meilleures performances pour la reconnaissance des chiffres manuscrits. Le modèle CNN amélioré a atteint une précision de test de 99.64%, démontrant ainsi l'efficacité des réseaux convolutifs dans l'extraction automatique des caractéristiques visuelles importantes. Le modèle MLP a également obtenu d'excellents résultats avec une précision de 99.19%, tandis que le modèle SVM a conservé des performances satisfaisantes avec une précision de 97.44%.

L'analyse des courbes d'apprentissage, des matrices de confusion ainsi que des métriques d'évaluation a permis de confirmer la bonne capacité de généralisation des modèles améliorés. Cette étude met également en évidence l'importance des techniques modernes de régularisation et d'optimisation dans l'amélioration des performances des systèmes de classification d'images.

Enfin, ce travail peut être étendu vers des applications plus complexes de reconnaissance d'images ou de caractères manuscrits en utilisant des architectures plus avancées telles que les réseaux de neurones profonds, les modèles ResNet ou encore les Transformers appliqués à la vision par ordinateur.

ANEXE

Le code source de ce projet est disponible sur GitHub : `https://github.com/RABAB-ENNACIRI/AI_project/blob/main/my_work_AI_project.ipynb`

Références

[1] R. Dixit, R. Kushwah, and S. Pashine, “Handwritten digit recognition using machine and deep learning algorithms,” *arXiv preprint arXiv :2106.12614*, June 2021.